

2024 SCHEME

KTUNOTES.LIVE

SCAN THE QR CODES

join our WhatsApp group



For WhatsApp group

get into our website



website for more materials

CLICK THE LOGO TO JOIN



OUR TELEGRAM CHANNEL

for notes and discussion and updates



OUR WHATSAPP GROUP

for note updates and notifications



OUR WEBSITE

get in and download all 2024 materials

MODULE 2

Microarchitecture

- Introduction;

Performance analysis;

Single-Cycle Processor - Single Cycle Data_path,

Single Cycle Control;

Pipelined Processor - Pipelined Data Path,

Pipelined Control: Hazards, Solving Data/Control Hazards,

Performance Analysis.

Introduction

- microarchitecture, which is the connection between logic and architecture.
- **Microarchitecture is the specific arrangement of registers, arithmetic logic units (ALUs), finite state machines (FSMs), memories, and other logic building blocks needed to implement an architecture.**
- A particular architecture, such as RISC-V, may have many different microarchitectures, each with different trade-offs of performance, cost, and complexity.
- They all run the same programs, but their internal designs vary widely

Architectural State and Instruction Set

- A computer architecture is defined by its instruction set and architectural state.
- The *architectural state* for the RISC-V processor consists of the program counter and the 32 32-bit registers.
- Any RISC-V microarchitecture must contain all of this state.
- Based on the current architectural state, the processor executes a particular instruction with a particular set of data to produce a new architectural state.

To keep the microarchitectures easy to understand, we focus on a subset of the RISC-V instruction set.

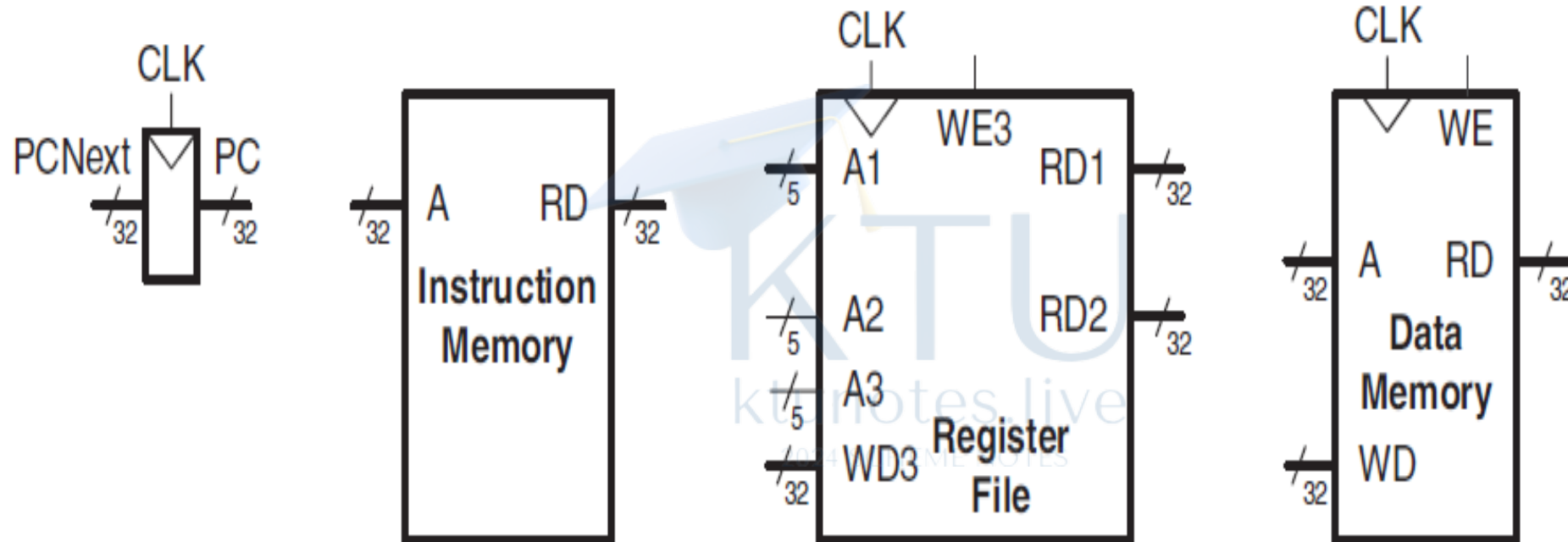
- ▶ R-type instructions: add, sub, and, or, slt
- ▶ Memory instructions: lw, sw
- ▶ Branches: beq

Design Process

- We divide microarchitectures into two interacting parts:
 - *data-path*
 - operates on words of data.
 - It contains structures such as memories, registers, ALUs, and multiplexers.
 - RISC-V (RV32I) architecture use a 32-bit data-path.
 - *control unit.*
 - The control unit receives the current instruction from the data-path and tells the data-path how to execute that instruction.
 - control unit produces multiplexer select, register enable, and memory write signals to control the operation of the data-path.

- A good way to design a complex system is to start with hardware containing the **state elements**.
- These elements include **the memories and the architectural state** (**the program counter and registers**).
- Then, add blocks of combinational logic between the state elements to compute the new state based on the current state.
- The instruction is read from part of memory; load and store instructions then read or write data from another part of memory.
- it is often convenient to partition the overall memory into two smaller memories, one containing instructions and the other containing data.

shows a block diagram with the **four state elements**: the **program counter**, **register file**, and **instruction and data memories**.



- The *program counter* (PC) points to the current instruction.
- Its input, *PCNext*, indicates the address of the next instruction.
- The ***instruction memory* has a single read port.**
- It takes a 32-bit instruction address input, *A*, and reads the 32-bit data (i.e., instruction) from that address onto the read data output, *RD*.
- The 32-element × 32-bit register file holds registers x0–x31.
- The register file has **two read ports and one write port.**
- The read ports take 5-bit address inputs, *A1* and *A2*, each specifying one of 32 registers as source operands.

Port	Description
A1	5-bit address of source register 1
A2	5-bit address of source register 2
RD1	32-bit data output from register A1
RD2	32-bit data output from register A2
A3	5-bit address of destination register
WD3	32-bit data to be written
WE3	Write enable
Clock	Controls write operation

Operation

- Read
 - Combinational (no clock needed).
 - Values from registers at addresses A1 and A2 appear on RD1 and RD2.
- Write
 - Occurs on the rising edge of the clock.
 - If $WE3 = 1$, data WD3 is written to register A3
 - If $A3 = x0$, write is ignored

- The register file places the 32-bit register values onto read data outputs *RD1* and *RD2*.
- The write port, port 3, takes a 5-bit address input, *A3*; a 32-bit write data input, *WD3*; a write enable input, *WE3*; and a clock.
- If its write enable (*WE3*) is asserted, then the register file writes the data (*WD3*) into the specified register (*A3*) on the rising edge of the clock.

data memory

- Stores data used by the program.
- Used by load (lw) and store (sw) instructions
- The *data memory* has a single read/write port.
- Address input: A
- Write data input: WD
- Read data output: RD
- Write enable: WE



Operation

■ Write Operation (Store Instruction)

- When $WE = 1$.
- On the rising edge of the clock
- The data WD is written into the memory location specified by address A .

$$\mathbf{Mem[A] \leftarrow WD}$$

■ Read Operation (Load Instruction)

- When $WE = 0$
- Memory performs a read
- The contents of memory at address A are placed onto the read data bus

$$\mathbf{RD \leftarrow Mem[A]}$$

- The instruction memory, register file, and data memory are all read *combinationally*..
- In other words, if the address changes, then the new data appears at *RD* after some propagation delay; no clock is involved.
- The clock controls writing only.
- These memories are written only on the rising edge of the clock.

Microarchitectures

- Three microarchitectures for the RISC-V architecture:
 - single-cycle
 - multicycle
 - pipelined.
- *single-cycle*
- The *single-cycle microarchitecture* executes an entire instruction in one cycle.
- It is easy to explain and has a simple control unit.
- cycle time is limited by the slowest instruction.
- Moreover, the processor requires separate instruction and data memories, which is generally unrealistic

multicycle

- The *multicycle microarchitecture* executes instructions in a series of shorter cycles.
- multicycle microarchitecture reduces the hardware cost by reusing expensive hardware blocks.
- The multicycle processor executes only one instruction at a time, but each instruction takes multiple clock cycles.

pipelined microarchitecture

- The *pipelined microarchitecture* applies pipelining to the single-cycle microarchitecture.
- It therefore can execute several instructions simultaneously, improving the throughput significantly.
- Pipelining must add logic to handle dependencies between simultaneously executing instructions.
- Pipelined processors must access instructions and data in the same cycle

PERFORMANCE ANALYSIS

- The cost of a processor architecture depends on the amount of hardware required and the implementation technology used.
- There are many ways to measure computer performance, such as clock frequency and number of cores.
- However, the most meaningful measure of performance is the execution time of a program.
- The computer that executes a program fastest is said to have the highest performance.
- When the actual program is not available, the total execution time of a set of similar programs is measured.
- Such collections of similar programs are called **benchmarks**.
- The execution times of benchmark programs are commonly published to provide an indication of how a processor performs.

- **Synthetic benchmarks**

- ✓ are artificial benchmark programs used to test processor performance.
- ✓ They consist of commonly used operations such as loops and arithmetic instructions.
- ✓ Examples: Dhrystone and CoreMark.

- **Real benchmarks**

- ✓ are composed of actual application programs.
- ✓ Example: SPECspeed 2017 Integer, which stresses the entire system realistically.
- ✓ The benchmark is widely used for high-performance processors because it stresses the entire system in a representative way.

Execution Time

$$ExecutionTime = (\#instructions) \left(\frac{cycles}{instruction} \right) \left(\frac{seconds}{cycle} \right)$$

- **Number of Instructions**

- ✓ Total instructions executed by the program
- ✓ depends on the algorithm, compiler, and ISA.

- **Cycles per Instruction (CPI)**

- ✓ Average number of clock cycles required to execute one instruction
- ✓ depends on processor design and implementation.
- ✓ It is the reciprocal of the throughput (*instructions per cycle*, or *IPC*).

- **Seconds per Cycle**

Represents the clock period, **denoted as T_c** .

$$T_c = \frac{1}{\text{Clock Frequency}}$$

The clock period is determined by the processor's clock frequency:

- Execution time depends on **instruction count**, **CPI**, and **clock period**.
- Reducing any one of these factors reduces the total execution time.
- Therefore, **higher clock frequency alone does not guarantee better performance**.
- The challenge of the micro architect is to choose the design that minimizes the execution time while satisfying constraints on cost and/or power consumption.
- Because micro architectural decisions affect both CPI and T_c and are influenced by logic and circuit designs, determining the best choice requires careful analysis.

SINGLE-CYCLE PROCESSOR

- Design a microarchitecture that executes instructions in a single cycle.
- begin constructing the data-path by connecting the state element with combinational logic that can execute the various instructions.
- Control signals determine which specific instruction is performed by the data-path at any given time.
- The control unit contains combinational logic that generates the appropriate control signals based on the current instruction.
- Finally, analyze the performance of the single-cycle processor

SINGLE-CYCLE PROCESSOR: Sample Program

- The program is stored in memory.
- starting at address 0x1000.
- The Program Counter (PC) initially contains 0x1000.
- The program demonstrates four RISC-V instruction formats: I-type, S-type, R-type, and B-type.
- Initial register and memory contents:
 - ✓ Register **x5 = 6**
 - ✓ Register **x9 = 0x2004**
 - ✓ Memory location **0x2000 = 10**

- Load instruction (lw):

- Calculates address: $0x2004 - 4 = 0x2000$
- Reads value 10 from memory
- Stores the value in register x6

- Store instruction (sw):

- Calculates address: $0x2004 + 8 = 0x200C$
- Writes value 10 into memory at address 0x200C

- **R-type instruction (or):**

- Performs bitwise OR

$$x4 = 6 \mid 10 = 0110_2 \mid 1010_2 = 1110_2 = 14$$

- **Branch instruction (beq):**

- Branches back to label **L7**
- Causes the program to **repeat indefinitely**

Address	Instruction	Type	Machine Language (Hex)	Action
0x1000	lw x6, -4(x9)	I	FFC4A303	Load value from $[\text{x9} - 4]$ into x6 .
0x1004	sw x6, 8(x9)	S	0064A423	Store value from x6 into $[\text{x9} + 8]$.
0x1008	or x4, x5, x6	R	0062E233	Bitwise OR of x5 and x6 into x4 .
0x100C	beq x4, x4, L7	B	FE420AE3	If $\text{x4} == \text{x4}$, jump back to L7 (0x1000).

SINGLE-CYCLE PROCESSOR: Sample Program

Address	Instruction	Type	Fields					Machine Language	
0x1000	L7: lw x6, -4(x9)	I	imm _{11:0}	rs1	f3	rd	op		
			111111111100	01001	010	00110	0000011	FFC4A303	
0x1004	sw x6, 8(x9)	S	imm _{11:5}	rs2	rs1	f3	imm _{4:0}	op	
			0000000	00110	01001	010	01000	0100011	0064A423
0x1008	or x4, x5, x6	R	funct7	rs2	rs1	f3	rd	op	
			0000000	00110	00101	110	00100	0110011	0062E233
0x100C	beq x4, x4, L7	B	imm _{12,10:5}	rs2	rs1	f3	imm _{4:1,11}	op	
			1111111	00100	00100	000	10101	1100011	FE420AE3

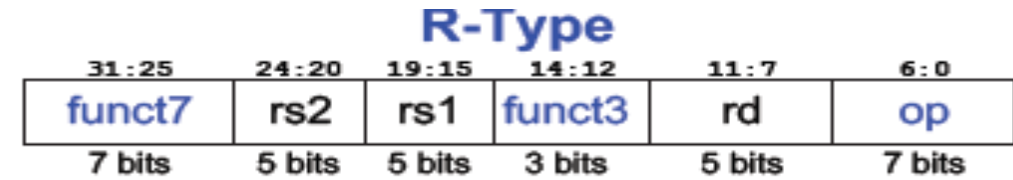
DEVELOPMENT OF THE SINGLE-CYCLE DATAPATH :

- This section explains the gradual development of the single-cycle datapath.
- New hardware components are added step by step to the state elements.
- New connections are highlighted in black (or blue for new control signals).
- Hardware already discussed is shown in gray.
- The example instruction being executed is shown at the bottom of each figure

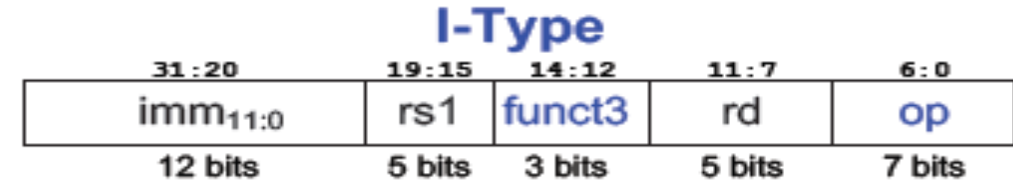
Instruction Fetch Step

- The Program Counter (PC) holds the address of the instruction to be executed.
- The first step in the data-path is instruction fetch.
- The PC output is directly connected to the address input of the instruction memory.
- The instruction memory reads (fetches) a 32-bit instruction, labeled Instr.
- Initial PC = 0x1000(Full 32-bit value is 0x00001000; leading zeros are omitted for clarity).
- Instruction memory outputs:
- Instr = lw
- Machine code: 0xFFC4A303

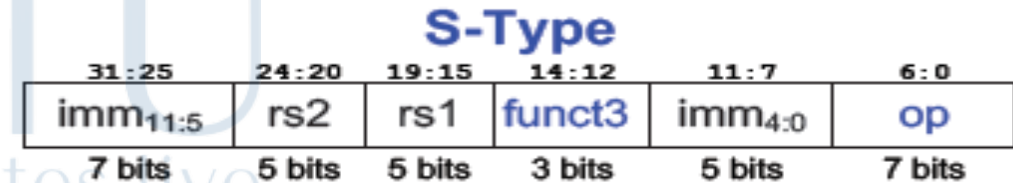
- **R-type instructions:**
 - **add, sub, and, or, slt**



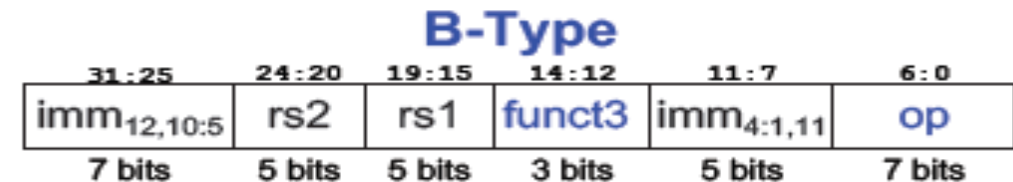
- **I-type instruction:**
 - **lw**



- **S-type instruction:**
 - **sw**



- **B-type instructions:**
 - **beq**



SINGLE-CYCLE PROCESSOR: SINGLE-CYCLE DATAPATH: LW FETCH

lw : load word instruction

STEP 1: Fetch instruction

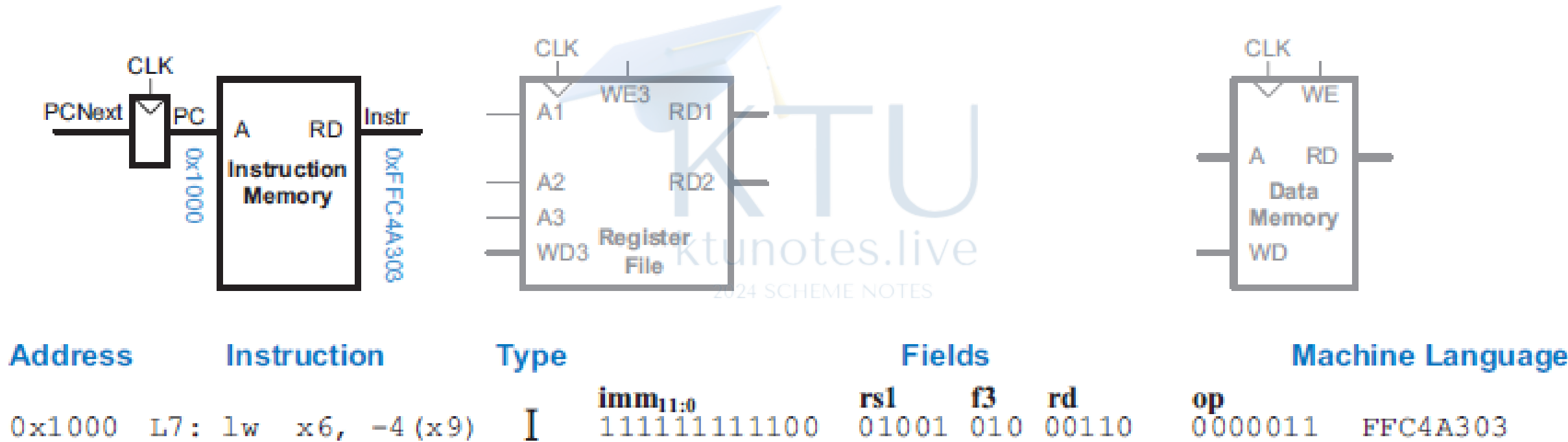


Figure 7.3 Fetch instruction from memory

SINGLE-CYCLE PROCESSOR: SINGLE-CYCLE DATA-PATH LW REG READ

STEP 2: Read source operand (rs1) from RF

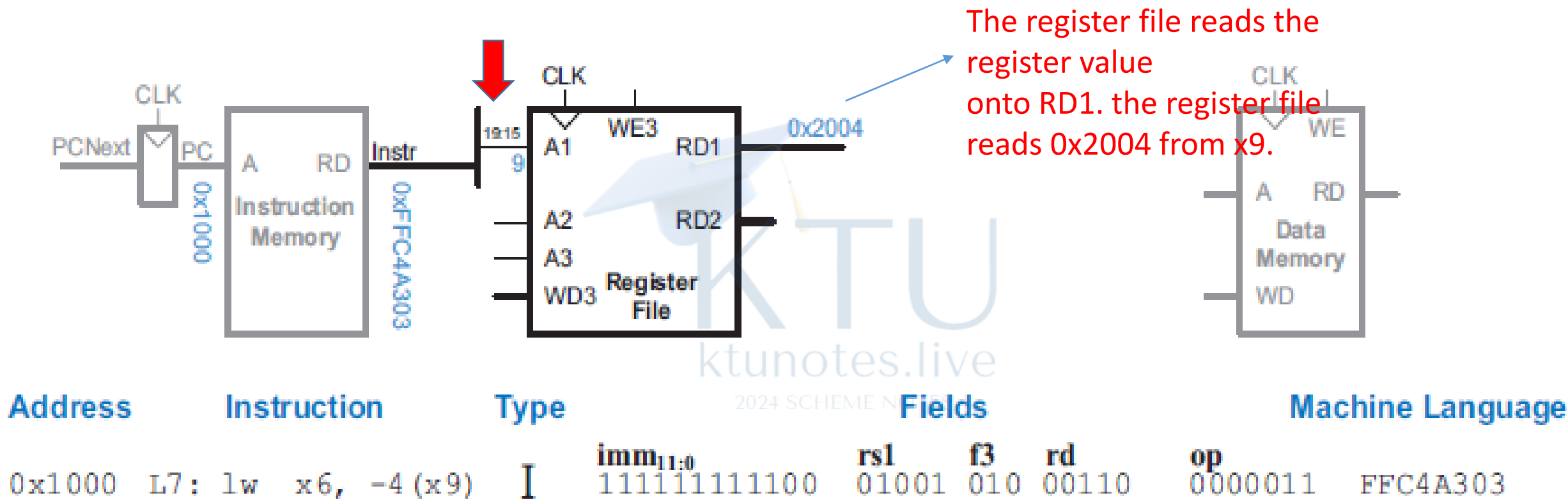


Figure 7.4 Read source operand from register file

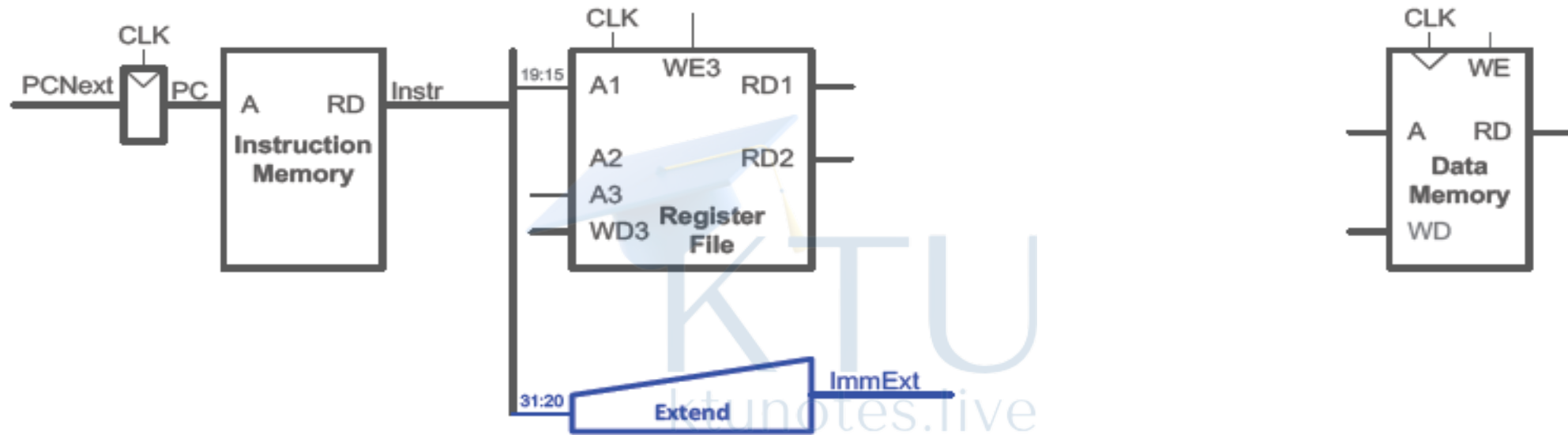
Instr19:15.

These bits of the instruction connect to the A1 address input of the register file

Single-Cycle Datapath: lw Immediate

- The lw instruction also requires an offset. The offset is stored in the 12-bit immediate field of the instruction, *Instr31:20*.
- It is a signed value, so it must be sign-extended to 32 bits. Sign extension simply means copying the sign bit into the most significant bits: $ImmExt31:12 = Instr31$, and $ImmExt11:0 = Instr31:20$.
- It is performed by an Extend unit.
- which receives the 12-bit signed immediate in *Instr31:20* and produces the 32-bit sign-extended immediate, *ImmExt*.
- In our example, the two's complement immediate -4 is 12-bit immediate -4 (0xFFC) and sign-extends it to 32 bits: 0xFFFFFFC.

STEP 3: Extend the immediate

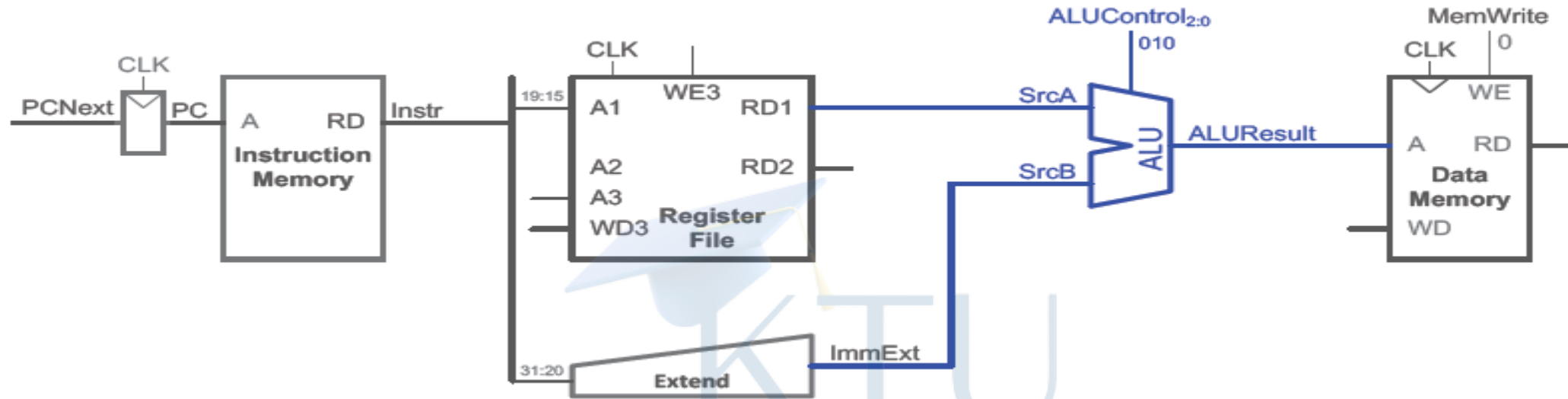


2024 SCHEME NOTES

Single-Cycle Datapath: lw Address

- The processor adds the base address to the offset to find the address to read from memory.
- introduces an ALU to perform this addition.
- The ALU receives two operands, *SrcA* and *SrcB*. *SrcA* is the base address from the register file, and *SrcB* is the offset from the sign-extended immediate, *ImmExt*.
- In our example, the ALU computes $0x2004 + 0xFFFFFFFFC = 0x2000$.
- Again, this is a 32-bit value, but we omit the leading zeros

STEP 4: Compute the memory address



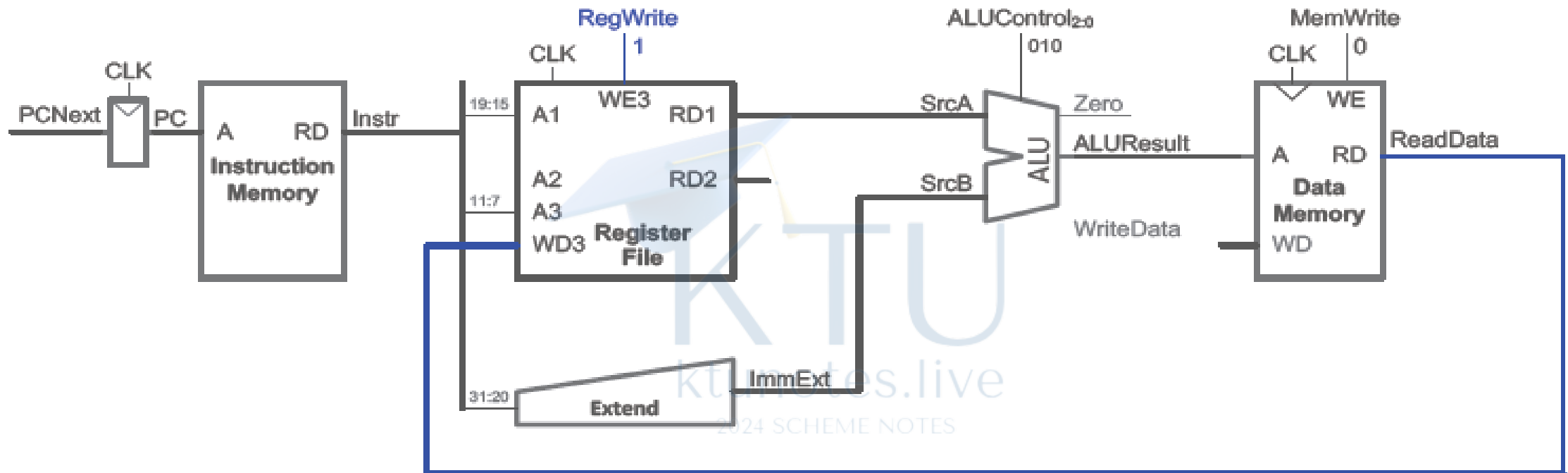
ktunotes.live

2024 SCHEME NOTES

Single-Cycle Datapath: LDR Mem Read

- This memory address from the ALU is provided to the address (*A*) port of the data memory.
- The data is read from the data memory onto the *ReadData* bus and then written back to the destination register at the end of the cycle.
- Port 3 of the register file is the write port.
- *lw*'s destination register, indicated by the **rd** field (*Instr*11:7), is connected to *A3*, port 3's address input.
- The *ReadData* bus is connected to *WD3*, port 3's write data input.
- A control signal called *RegWrite* (register write) is connected to *WE3*, port 3's write enable input, and is asserted during the *lw* instruction so that the data value is written into the register file.
- In our example, the processor reads 10 from address 0x2000 in the data memory and puts that value (10) into x6 in the register file.

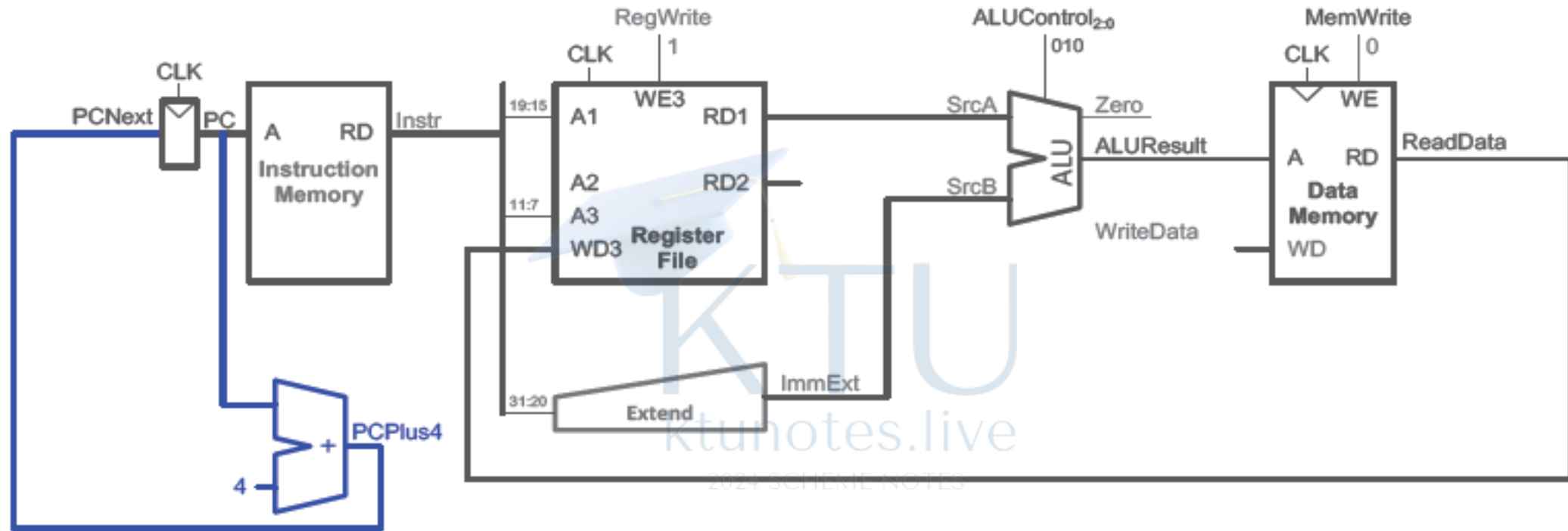
STEP 5: Read data from memory and write it back to register file



Single-Cycle Datapath: PC Increment

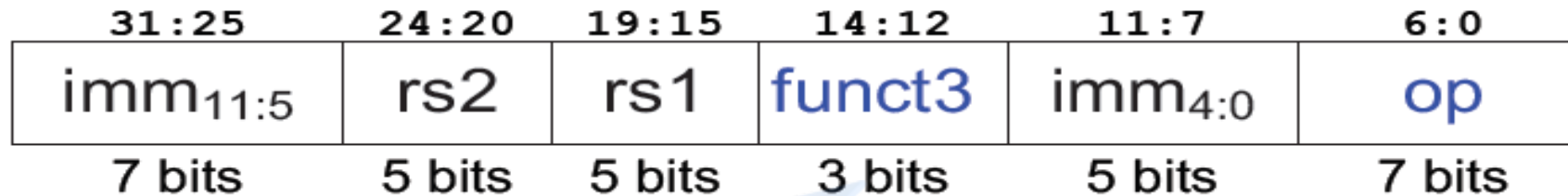
- While the instruction is being executed, the processor must also compute the address of the next instruction, *PCNext*.
- Because instructions are 32 bits (4 bytes), the next instruction is at PC+4.
- In our example, $PCNext = 0x1000 + 4 = 0x1004$.
- The new address is written into the program counter on the next rising edge of the clock.
- This completes the datapath for the lw instruction.

STEP 6: Determine address of next instruction



Single-Cycle Datapath: sw

S-Type

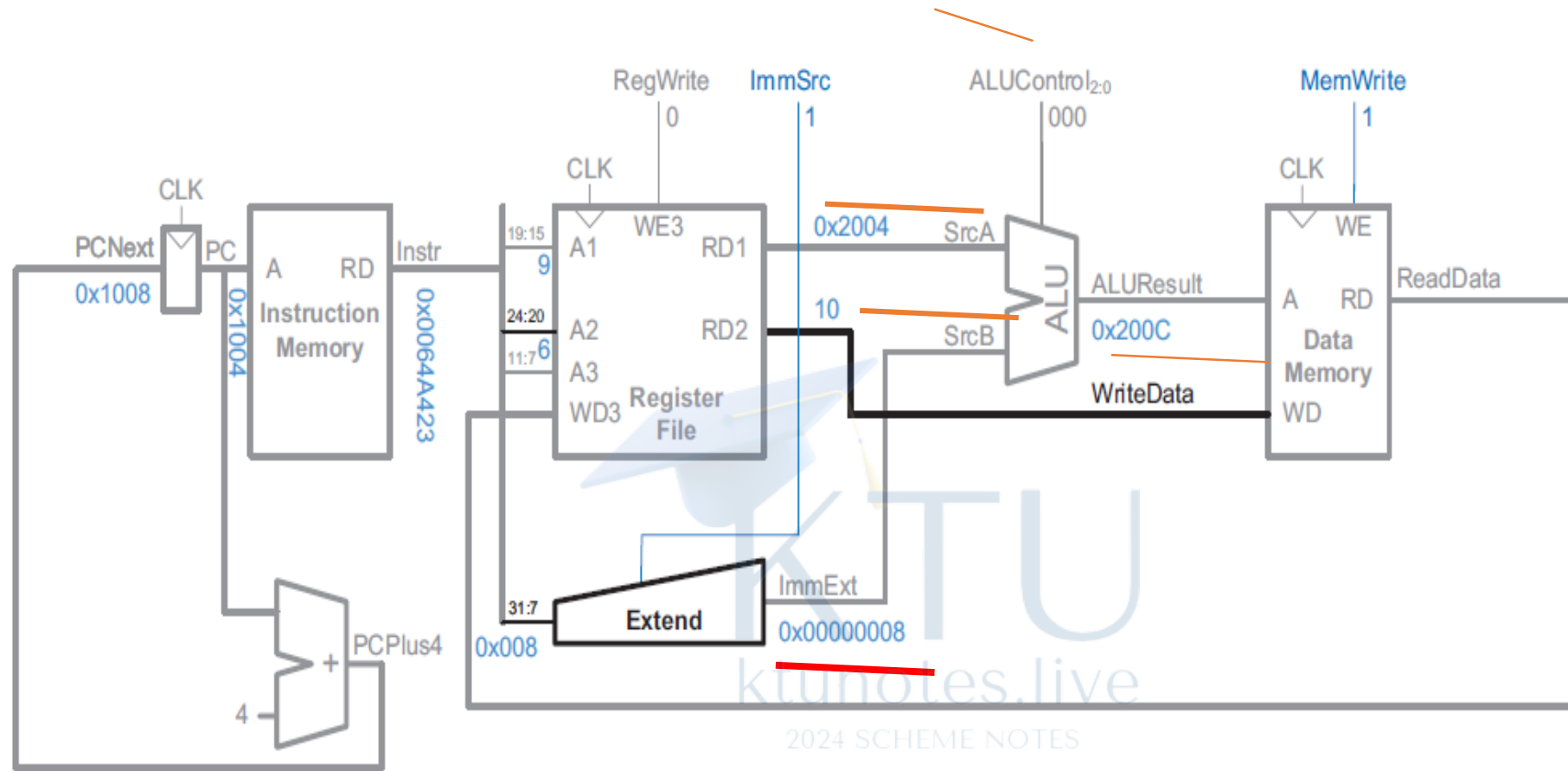


- sw, which is an S-type instruction.
- Like lw, sw reads a base address from port 1 of the register file and sign-extends the immediate.
- The ALU adds the base address to the immediate to find the memory address.
- All of these functions are already supported in the datapath, but the 12-bit signed immediate is stored in Instr31:25,11:7 (instead of Instr31:20, as it was for lw).
- Thus, the Extend unit must be modified to also receive these additional bits, Instr11:7.

- Thus, the Extend unit must be modified to also receive these additional bits, ***Instr11:7***.
- For simplicity (and for future instructions such as jal), the Extend unit receives all the bits of *Instr31:7*.
- A control signal, ***ImmSrc***, which instruction bits to use as the immediate.
- **When *ImmSrc* = 0 (for lw), the Extend unit chooses *Instr31:20* as the 12-bit signed immediate;**
- **when *ImmSrc* = 1 (for sw), it chooses *Instr31:25,11:7*.**

ImmSrc	ImmExt	Instruction Type
0	{{20{instr[31]}}, instr[31:20]}	I-Type
1	{{20{instr[31]}}, instr[31:25], instr[11:7]}	S-Type

- The sw instruction also reads a second register from the register file and writes its contents to the data memory.
- The register is specified in the rs2 field, **Instr24:20**, which is connected to the address **2 (A2)** input of the register file.
- The register's contents are read onto the read data **2 (RD2) output**, which, in turn, is connected to the **write data (WD)** input of the data memory.
- The write enable port of the data memory, WE, is controlled by **MemWrite**.
- For an sw instruction: **MemWrite = 1 to write the data to memory;**
- **ALUControl = 000 to add the base address and offset; and RegWrite = 0, because nothing should be written to the register file.**
- Note that data is still read from the address given to the data memory, but this *ReadData* is ignored because *RegWrite* = 0.



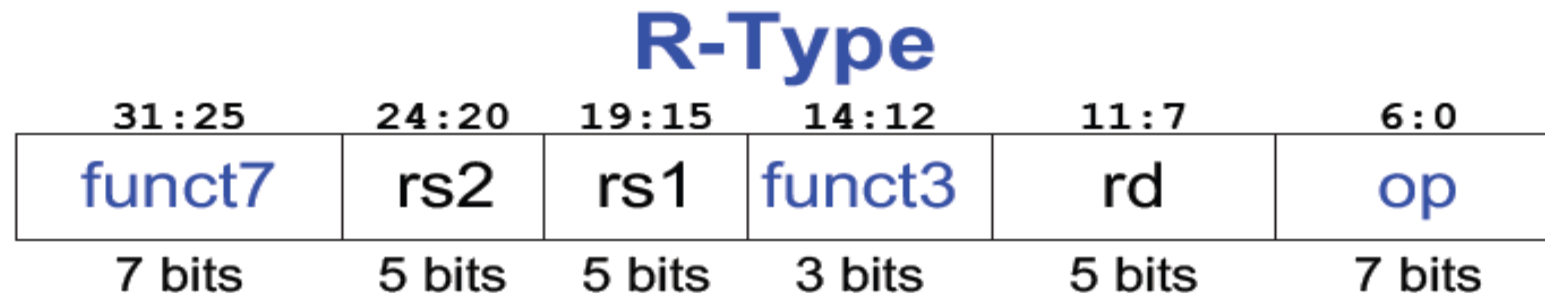
Address	Instruction	Type	Fields					Machine Language	
0x1004	sw x6, 8(x9)	S	imm _{11:5}	rs2	rs1	f3	imm _{4:0}	op	0064A423
			0000000	00110	01001	010	01000	0100011	

Figure 7.9 Write data to memory for sw instruction

- In our example, the PC is 0x1004. -- **sw x6, 8(x9)**
 - Thus, the instruction memory reads out the sw instruction, 0x0064A423.
 - The register file reads 0x2004 (the base address) from x9 and 10 from x6 while the Extend unit extends the immediate offset 8 from 12 to 32 bits.
 - The ALU computes $0x2004 + 8 = 0x200C$.
 - The data memory writes 10 to address 0x200C.
 - Meanwhile, the PC is incremented to 0x1008.

Single-Cycle Datapath: R-Type

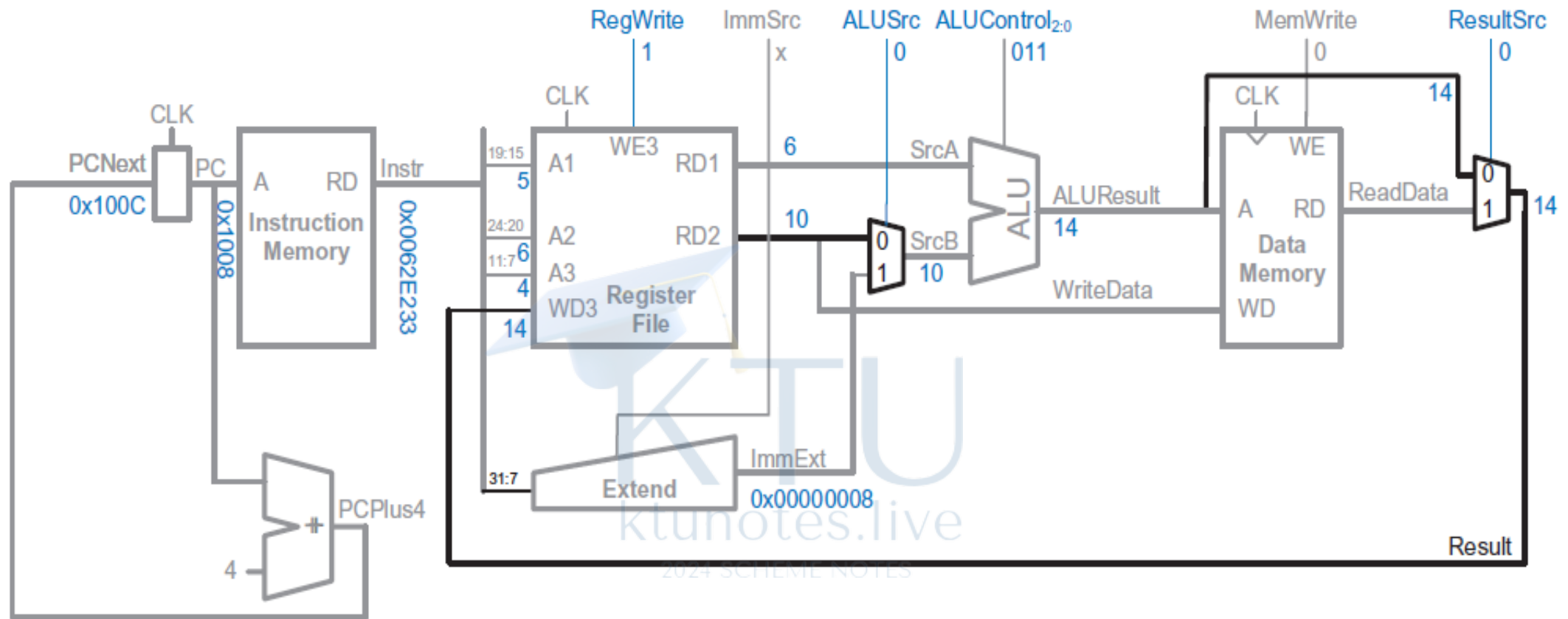
- **Instructions:** add, sub, and, or, slt.
- All of these instructions read two source registers from the register file, perform some ALU operation on them, and write the result back to the destination register.
- They differ only in the specific ALU operation.
- Hence, they can all be handled with the same hardware but with different *ALUControl* signals.



- ✓ *ALUControl* = 000 for addition.
- ✓ *ALUControl* = 001 for subtraction
- ✓ *ALUControl* = 010 for AND.
- ✓ *ALUControl* = 011 for OR.
- ✓ *ALUControl* = 101 for less than.

Single-Cycle Datapath: R-Type

- The datapath reads **rs1** and **rs2** from ports 1 and 2 of the register file and performs an ALU operation on them.
- Write *ALUResult* to **rd**
- We introduce **a multiplexer and a new select signal, *ALUSrc*, to select between *ImmExt* and *RD2*** as the second ALU source, *SrcB*.
- **For lw and sw, *ALUSrc* is 1 to select *ImmExt*;**
- **for R-type instructions, *ALUSrc* is 0 to select the register file output *RD2* as *SrcB*.**



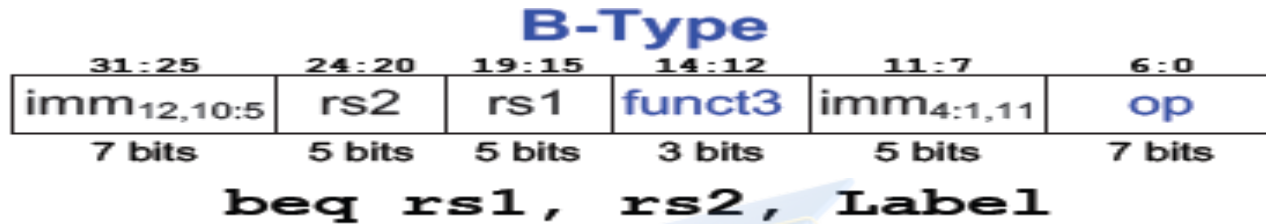
Address	Instruction	Type	Fields					Machine Language	
			funct7	rs2	rs1	f3	rd	op	
0x1008	or x4, x5, x6	R	0000000	00110	00101	110	00100	0110011	0062E233

Figure 7.10 Datapath enhancements for R-type instructions

- We add the Result multiplexer to choose the proper *Result* based on the type of instruction.
- The multiplexer select signal
 - **ResultSrc is 0** for R-type instructions to choose *ALUResult* as *Result*;
 - **ResultSrc is 1** for *lw* to choose *ReadData*.
- We do not care about the value of *ResultSrc* for *sw* because it does not write the register file.
- In our example, **or x4, x5, x6**
 - the PC is 0x1008.
 - Thus, the instruction memory reads out the *or* instruction 0x0062E233.
 - The register file reads source operands 6 from x5 and 10 from x6.
 - *ALUControl* is 011, so the ALU computes $6 \mid 10 = 0110_2 \mid 1010_2 = 1110_2 = 14$.
 - The result is written back to x4.
 - Meanwhile, the PC is incremented to 0x100C.

Single-Cycle Datapath: beq

- Calculate branch target address: $PCTarget = PC + imm$.



- beq compares two registers.
- If they are equal, it takes the branch by adding the branch offset to the program counter (PC).
- The branch offset is a 13-bit signed immediate stored in the 12-bit immediate field of the B-type instruction.
- *ImmSrc* is increased to 2 bits, using the encoding from Table 7.1.
- *ImmExt* is now either the sign-extended immediate (when *ImmSrc* = 00 or 01) or the branch offset (when *ImmSrc* = 10).

ImmSrc _{1:0}	ImmExt	Instruction Type
00	{{20{instr[31]}}, instr[31:20]}	I-Type
01	{{20{instr[31]}}, instr[31:25], instr[11:7]}	S-Type
10	{{19{instr[31]}}, instr[31], instr[7], instr[30:25], instr[11:8], 1'b0}	B-Type

I-Type



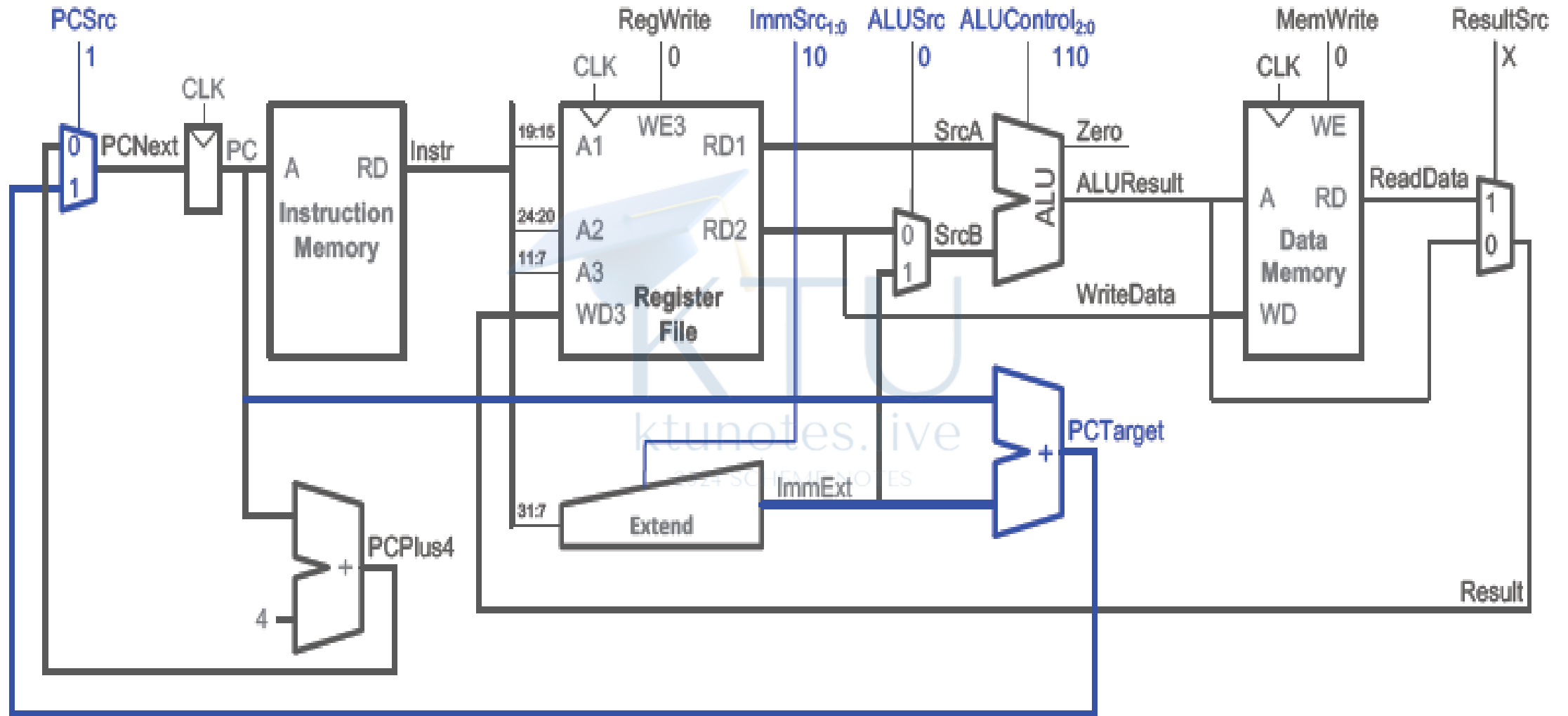
S-Type



B-Type

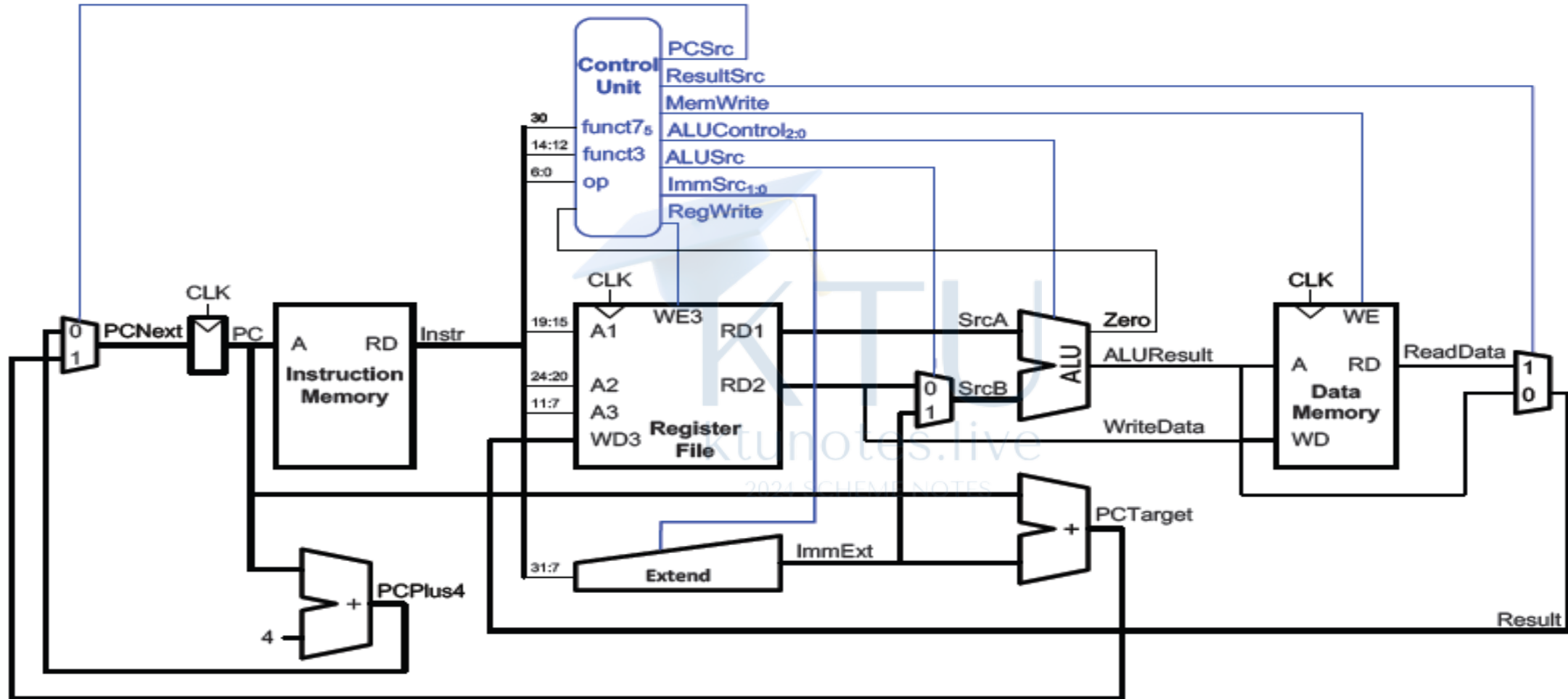


- The two source registers are compared by computing $(SrcA - SrcB)$ using the ALU.
- If $ALUResult$ is 0, as indicated by the ALU's Zero flag, the registers are equal.
- We add a multiplexer to choose $PCNext$ from either $PCPlus4$ or $PCTarget$.
- $PCTarget$ is selected if the instruction is a branch and the Zero flag is asserted.
- For beq , $ALUControl = 001$, so that the ALU performs a subtraction.
- $ALUSrc = 0$ to choose $SrcB$ from the register file.
- $RegWrite$ and $MemWrite$ are 0, because a branch does not write to the register file or memory.

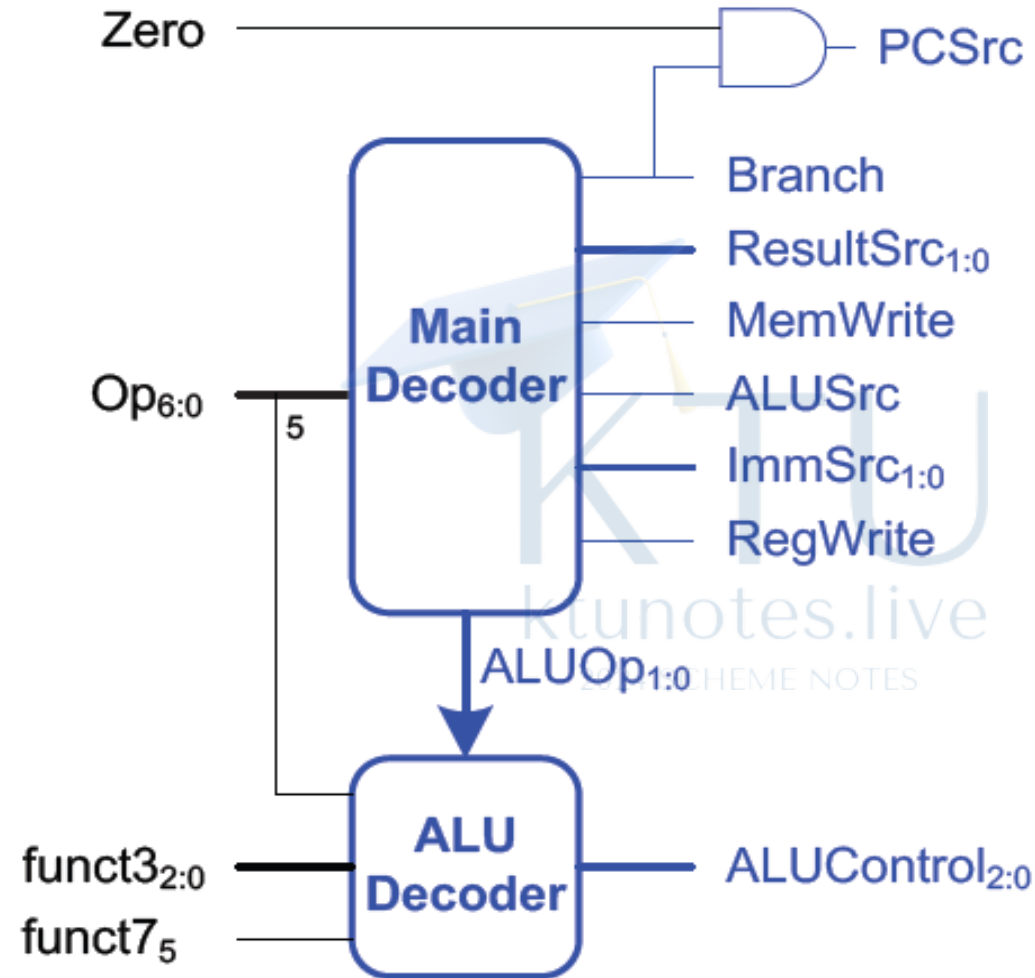


- In our example, **beq x4, x4, L7**
 - the PC is 0x100C, so the instruction memory reads out the beq instruction 0xFE420AE3.
 - Both **source registers are x4, so the register** file reads 14 on both ports.
 - The ALU computes $14 - 14 = 0$, and the **Zero flag is asserted**. Meanwhile, the Extend unit produces 0xFFFFFFFF4 (i.e., -12), which is added to *PC* to obtain *PCTarget* = 0x1000.
 - The PCNext mux chooses *PCTarget* as the next PC and branches back to the start of the code at the next clock edge.

Single-Cycle RISC-V Processor



Single-Cycle Control



Single-Cycle Control

- The single-cycle processor's control unit computes the control signals based on **op**, **funct3**, and **funct7**.
- the control unit, which is also referred to as the *controller* or the *decoder*, because it decodes what the instruction should do.
- We partition it into two major parts:
 - the **Main Decoder**, which produces most of the control signals, &
 - the **ALU Decoder**, which determines what operation the ALU performs.

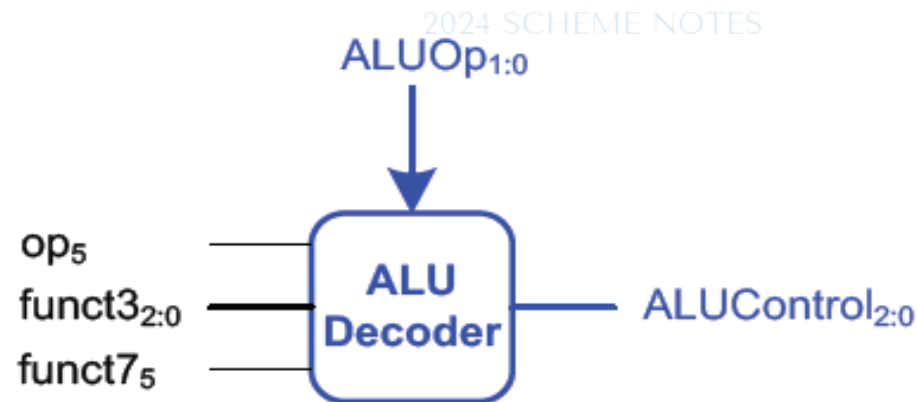
Instruction	Op	RegWrite	ImmSrc	ALUSrc	MemWrite	ResultSrc	Branch	ALUOp
lw	0000011	1	00	1	0	1	0	00
sw	0100011	0	01	1	1	x	0	00
R-type	0110011	1	xx	0	0	0	0	10
beq	1100011	0	10	0	0	x	1	01

- Table shows the control signals that the Main Decoder produces.
- The Main Decoder determines the instruction type from the opcode and then produces the appropriate control signals for the datapath.
- The Main Decoder generates most of the control signals for the datapath.
- It also produces internal signals *Branch* and *ALUOp*, signals used within the controller.

- The ALU Decoder produces *ALUControl* based on *ALUOp* and **funct3**.
- In the case of the sub and add instructions, the ALU Decoder also uses **funct7** and **op5** to determine *ALUControl* as given in in Table.

ALUOp	funct3	{op ₅ , funct ₇ }	ALUControl	Instruction
00	x	x	000 (add)	lw, sw
01	x	x	001 (subtract)	beq
10	000	00, 01, 10	000 (add)	add
	000	11	001 (subtract)	sub
	010	x	101 (set less than)	slt
	110	x	011 (or)	or
	111	x	010 (and)	and

- *ALUOp* of 00 indicates add (e.g., to find the address for loads or stores).
- *ALUOp* of 01 indicates subtract (e.g., to compare two numbers for branches).
- *ALUOp* of 10 indicates an R-type ALU instruction where the ALU Decoder must look at the **funct3** field (and sometimes also the **op5** and **funct75** bits) to determine which ALU operation to perform (e.g., add, sub, and, or, slt).



Example: or

op	Instruct	RegWrite	ImmSrc	ALUSrc	MemWrite	ResultSrc	Branch	ALUOp
51	R-type	1	XX	0	0	0	0	10

KTU

ktunotes.live

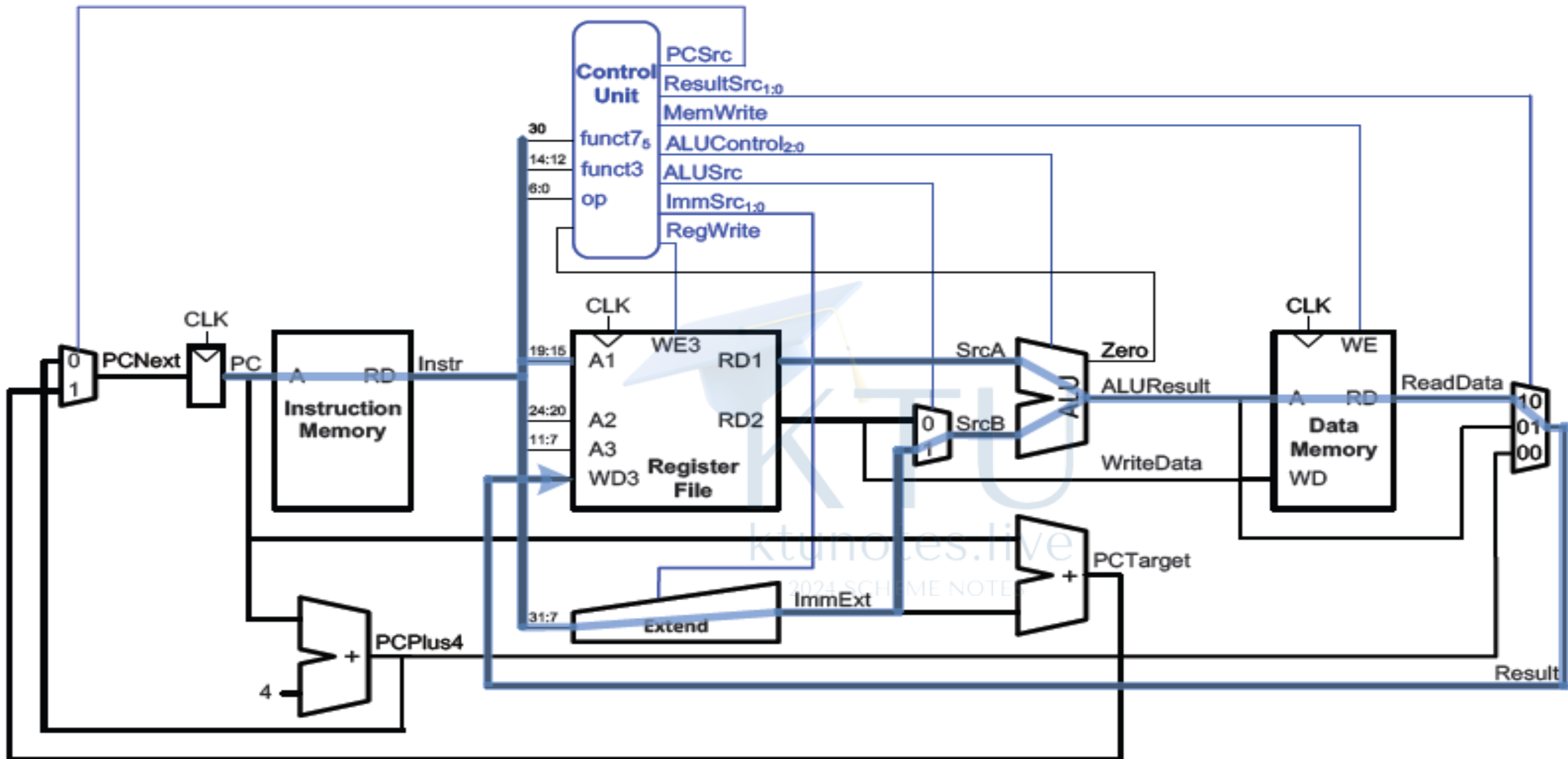
2024 SCHEME NOTES

Single-Cycle Performance

- The execution time of a program is the product of the number of instructions, the cycles per instruction, and the cycle time.
- Each instruction in the single-cycle processor takes one clock cycle, so the clock cycles per instruction (CPI) is 1.
- The cycle time is set by the critical path. In our processor, the lw instruction is the most time-consuming and involves the critical path.

ktunotes.live

2024 SCHEME NOTES



T_c limited by critical path (1w)

- Hence, the cycle time of the single-cycle processor is

$$T_{c_single} = t_{pcq_PC} + t_{mem} + \max[t_{RFread}, t_{dec} + t_{ext} + t_{mux}] \\ + t_{ALU} + t_{mem} + t_{mux} + t_{RFsetup}$$

- In most implementation technologies, the ALU, memory, and register file are substantially slower than other combinational blocks. Therefore, the critical path is through the register file—not through the decoder (controller), Extend unit, and multiplexer.

$$T_{cl} = t_{pcq_PC} + 2t_{mem} + t_{RFread} + t_{ALU} + 2t_{mux} + t_{RFsetup}$$

- **Single-cycle critical path:**

$$T_{cl} = t_{pcq_PC} + t_{mem} + \max[t_{mux} + t_{RFread}, t_{ext} + t_{mux}] + t_{ALU} + t_{mem} + t_{mux} + t_{RFsetup}$$

- **Typically, limiting paths are:**

- memory, ALU, register file

- $T_{cl} = t_{pcq_PC} + 2t_{mem} + t_{RFread} + t_{ALU} + 2t_{mux} + t_{RFsetup}$



Single-Cycle Performance Example

Element	Parameter	Delay (ps)
Register clock-to-Q	t_{pcq_PC}	40
Register setup	t_{setup}	50
Multiplexer	t_{mux}	25
ALU	t_{ALU}	120
Decoder (Control Unit)	t_{dec}	70
Memory read	t_{mem}	200
Register file read	t_{RFread}	100
Register file setup	$t_{RFsetup}$	60

$$\begin{aligned}T_{cl} &= t_{pcq_PC} + 2t_{mem} + t_{dec} + t_{RFread} + t_{ALU} + 2t_{mux} + t_{RFsetup} \\&= [50 + 2(200) + 70 + 100 + 120 + 2(25) + 60] \text{ ps} \\&= \mathbf{840 \text{ ps}}\end{aligned}$$

Single-Cycle Performance Example

Program with 100 billion instructions:

$$\begin{aligned}\text{Execution Time} &= \# \text{ instructions} \times \text{CPI} \times T_C \\ &= (100 \times 10^9)(1)(840 \times 10^{-12} \text{ s}) \\ &= \mathbf{84 \text{ seconds}}\end{aligned}$$

2024 SCHEME NOTES

The cycle time of the single-cycle processor is 750 ps .Compute the execution time for a program with 20 billion instructions.

Part 2

- Pipelined Processor - Pipelined Data Path,
- Pipelined Control: Hazards, Solving Data/Control Hazards, Performance Analysis.

ktunotes.live

2024 SCHEME NOTES

PIPELINED PROCESSOR

- Thus, five instructions can execute simultaneously, one in each stage.
- The five stages are Fetch, Decode, Execute, Memory, and Writeback.

PIPELINED PROCESSOR

- Pipelining is a powerful way to **improve the throughput** of a digital system.
- In a RISC-V processor, the execution of an instruction is broken down into five distinct stages.
- We design a pipelined processor by subdividing the **single-cycle processor into five pipeline stages**.
- Thus, five instructions can execute simultaneously, one in each stage.
- Because **each stage has only one-fifth** of the entire logic, the clock frequency is approximately five times faster.
- So, ideally, **the latency of each instruction is unchanged**, but the **throughput is five times better**.
- Microprocessors execute millions or billions of instructions per second, so **throughput is more important than latency**.

PIPELINED PROCESSOR

- Reading and writing the memory and register file and using the ALU typically constitute the biggest delays in the processor.
- We choose five pipeline stages so that each stage involves exactly one of these slow steps.
- The five stages are **Fetch, Decode, Execute, Memory, and Writeback**.
- They are similar to the five steps that the multicycle processor used to perform lw like operations.

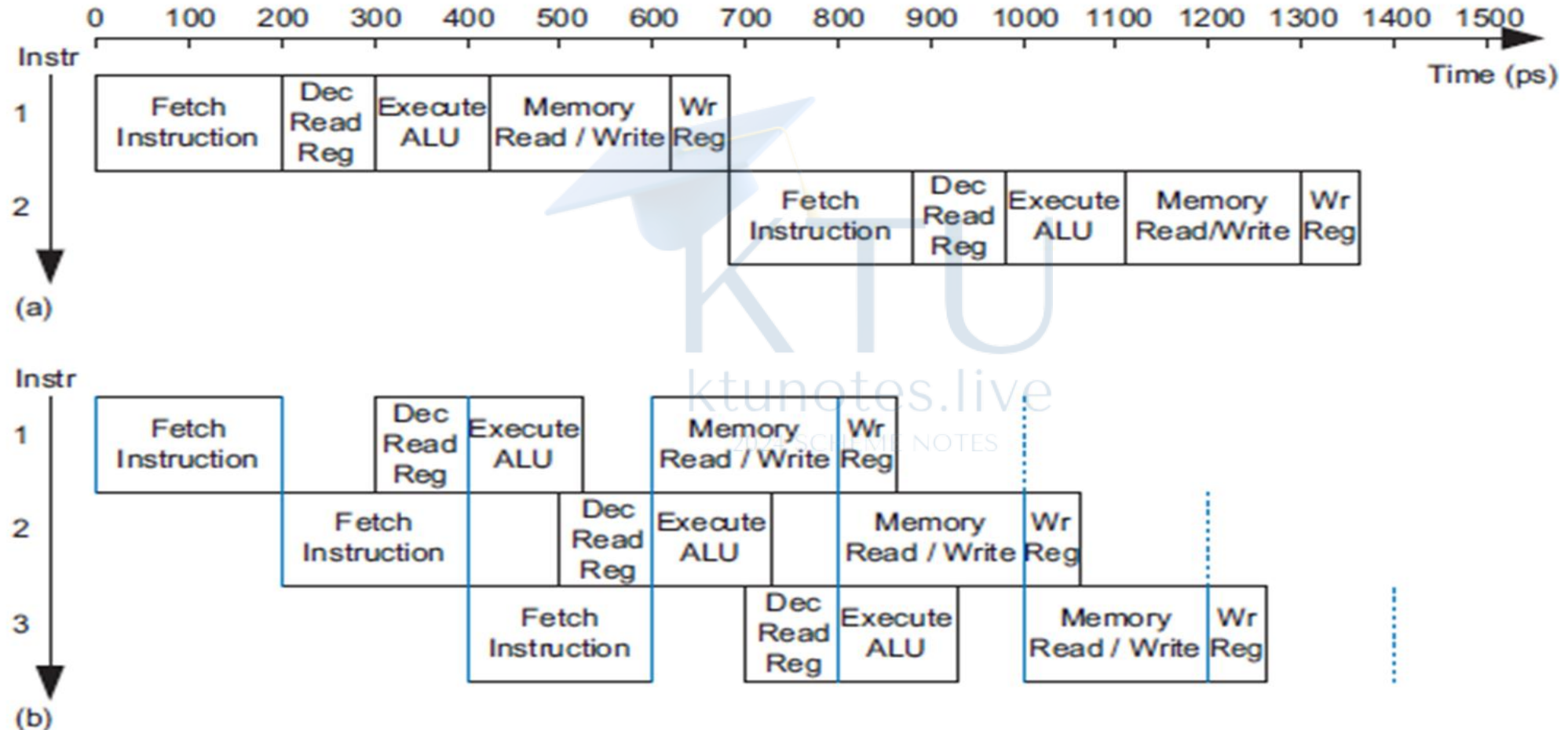
ktunotes.live

2024 SCHEME NOTES

- In the *Fetch* stage, the processor reads the instruction from instruction memory.
- In the *Decode* stage, the processor reads the source operands from the register file and decodes the instruction to produce the control signals.
- In the *Execute* stage, the processor performs a computation with the ALU.
- In the *Memory* stage, the processor reads or writes data memory, if applicable.
- Finally, in the *Writeback* stage, the processor writes the result to the register file, if applicable.

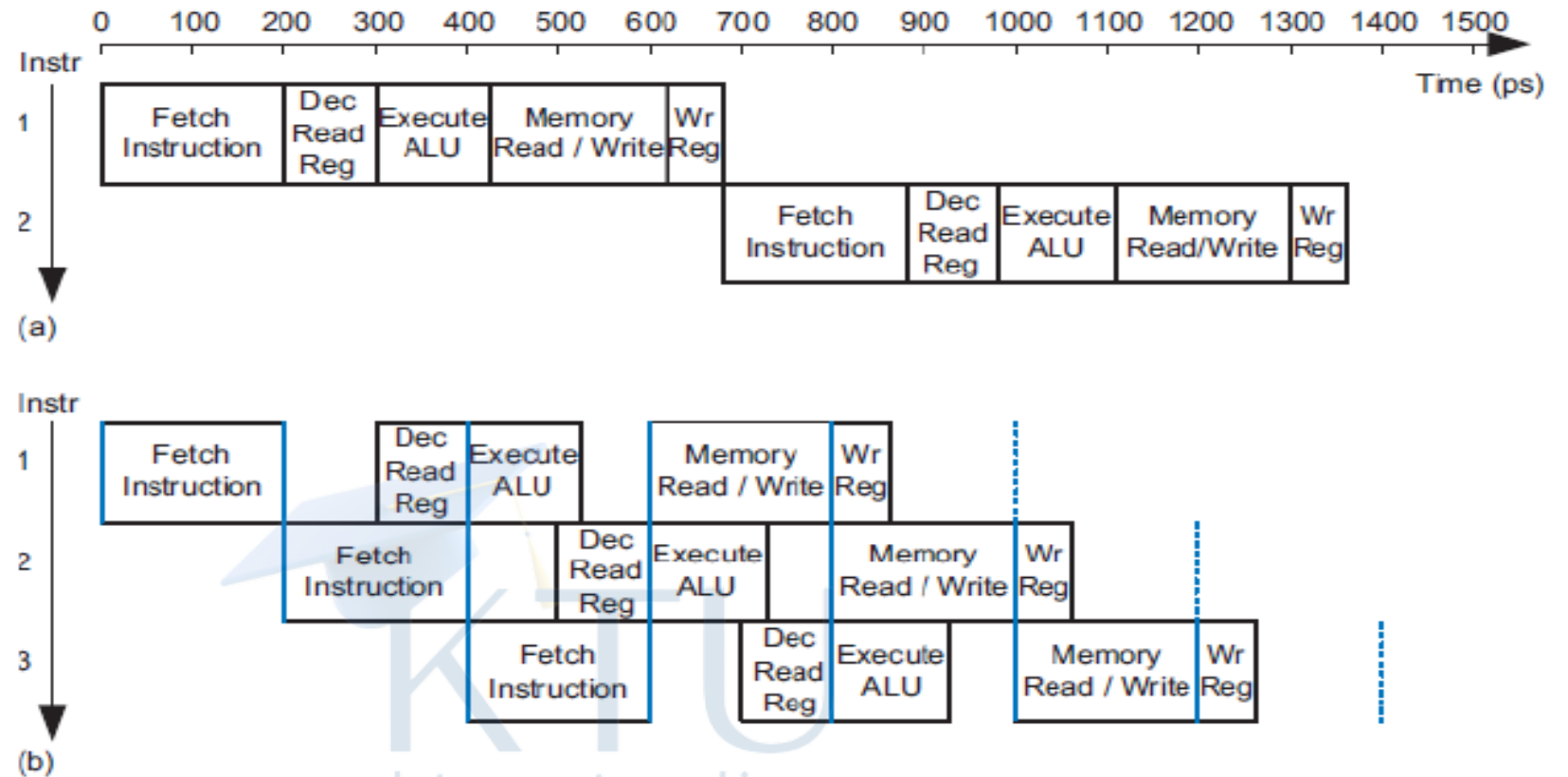
Timing diagram comparing the single-cycle and pipelined processors.

- Time is on the horizontal axis and instructions are on the vertical axis.



- Delay or Latency in each Stages

- Fetch - 200ps
- Decode – 100 ps
- Execute – 120 ps
- Memory R/W - 200 ps
- Write Reg – 60 ps



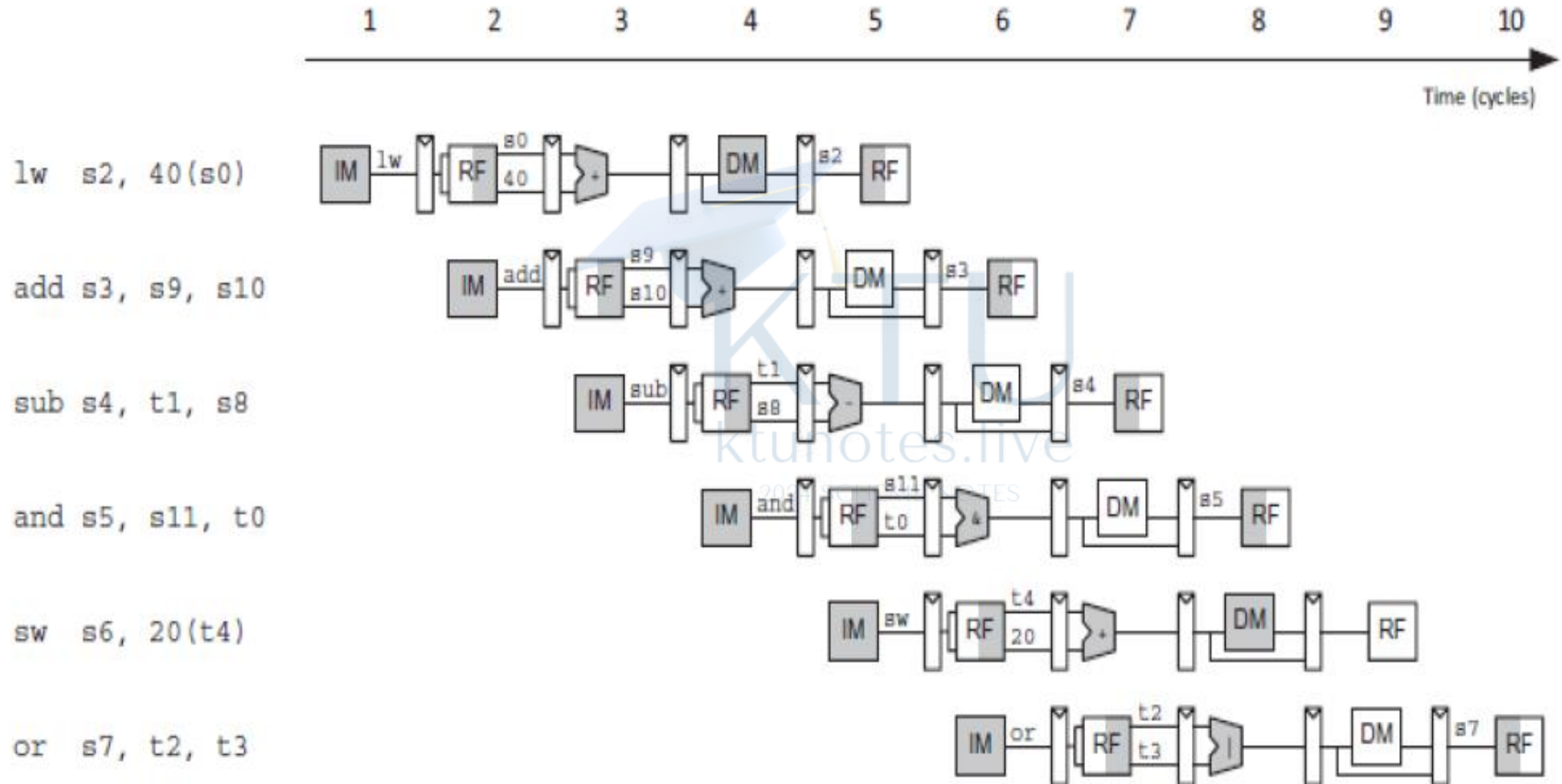
In the pipelined processor the length of a pipeline stage is set at 200 ps by the slowest stage, the memory access in the Fetch or Memory stage

- the length of a pipeline stage is **set at 200 ps by the slowest stage**, the memory access in the Fetch or Memory stage.
- The instruction latency is $5 \times 200 = 1000$ ps.
- The throughput is 1 instruction per 200 ps (5 billion instructions per second)—that is, one instruction completes every clock cycle.
- This throughput is 3.4 times as much as the single-cycle processor

ktunotes.live

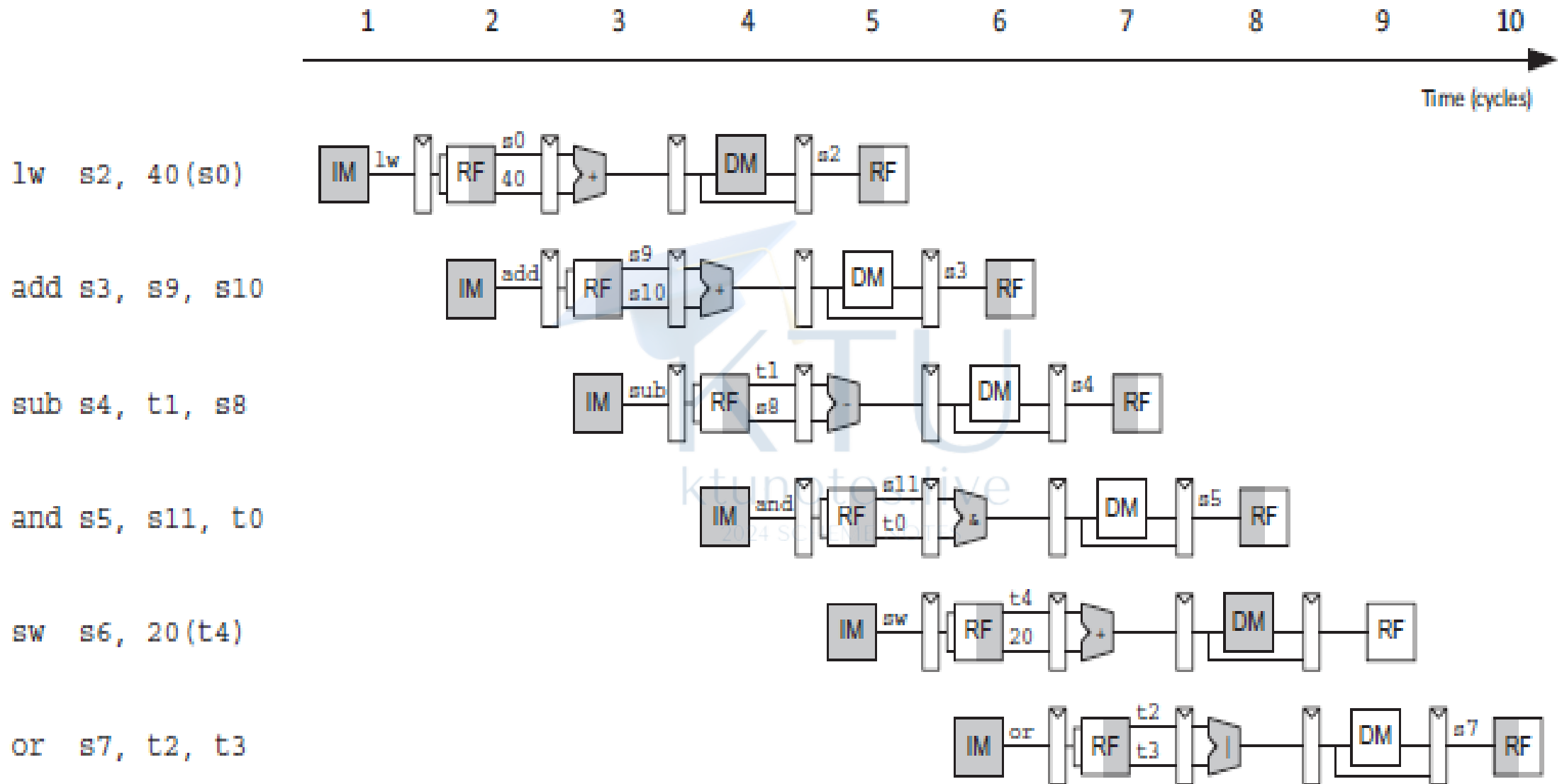
2024 SCHEME NOTES

Abstract view of pipeline in operation



Abstract view of pipeline in operation

- Each pipeline stage is represented with its major component
 - instruction memory (IM),
 - register rfile (RF) read,
 - ALU execution
 - data memory (DM)
 - register file write_back —to illustrate the flow of instructions through the pipeline.
- Reading across a row shows the clock cycle in which a particular instruction is in each stage.
- In the pipelined processor, the register file is used twice in every cycle: it is written in the first part of a cycle and read in the second part.
- This way, data can be written by one instruction and read by another within a single cycle.
- A central challenge in pipelined systems is handling **hazards** that occur when one instruction's result is needed by a subsequent instruction before the former instruction has completed

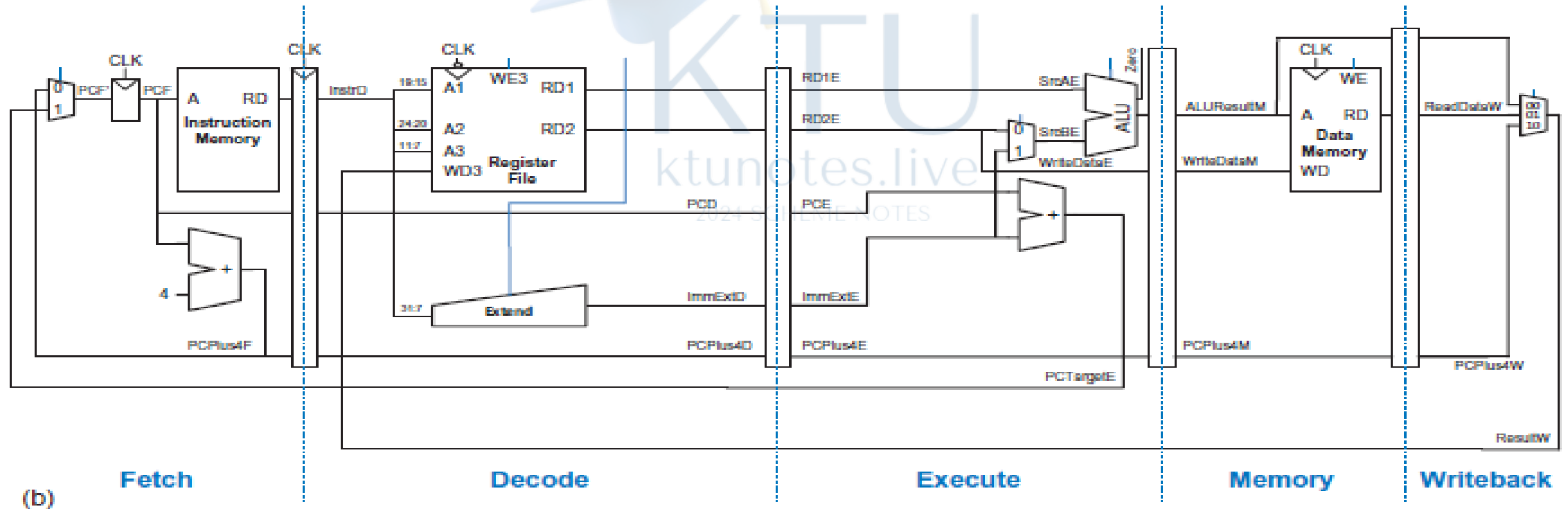
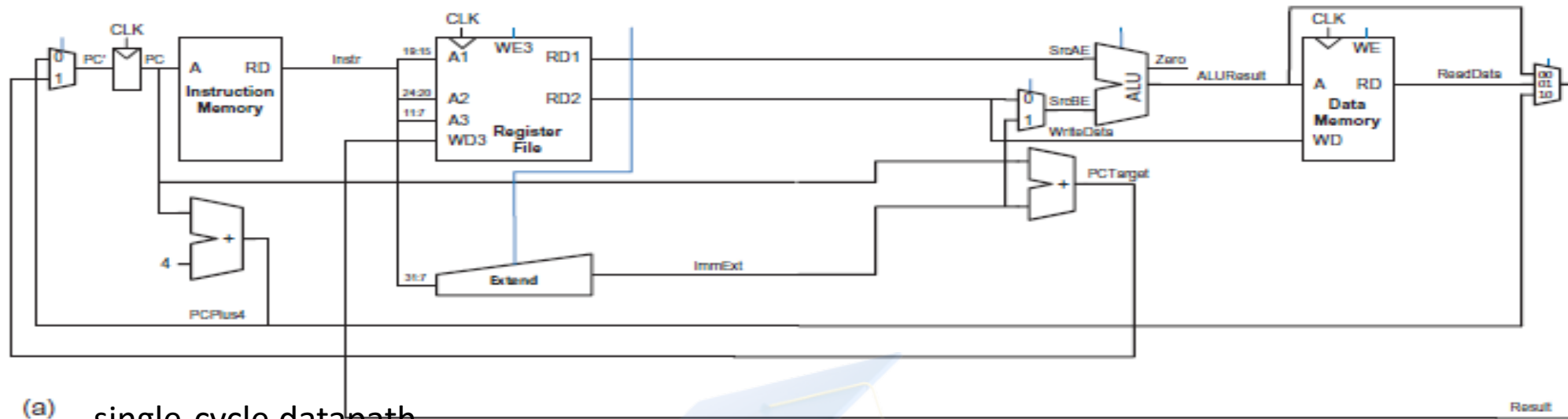


Pipeline Processor Data-path

- The pipelined data-path is formed by chopping the single-cycle data_path into five stages separated by pipeline registers.
- figure (a) shows the single-cycle datapath
- Figure (b) shows the pipelined datapath formed by inserting four pipeline registers to separate the datapath into five stages.
- Signals are given a suffix (F, D, E, M, or W) to indicate the stage in which they reside.

ktunotes.live

2024 SCHEME NOTES



- The register file is peculiar because it is read in the Decode stage and written in the Writeback stage.
- So, although the register file is drawn in the Decode stage, its write address and write data come from the Writeback stage.
- This feedback will lead to pipeline hazards.
- The register file in the pipelined processor writes on the falling edge of *CLK* so that it can write a result in the first half of a cycle and read that result in the second half of the cycle for use in a subsequent instruction.

ktunotes.live

2024 SCHEME NOTES

- The error is in the register file write logic, which should operate in the Write back stage.
- The data value comes from *ResultW*, a Writeback stage signal. But the destination register comes from *RdD* (*InstrD11:7*) which is a Decode stage signal.
- In the pipeline diagram of Figure during cycle 5, the result of the lw instruction would be incorrectly written to s5 rather than s2.

ktunotes.live

2024 SCHEME NOTES

- With the modification -
 - ✓ The *Rd* signal is now pipelined along through the Execution, Memory, and Write back stages, so it remains in sync with the rest of the instruction.
 - *RdW* and *ResultW* are fed back together to the register file in the Write back stage.

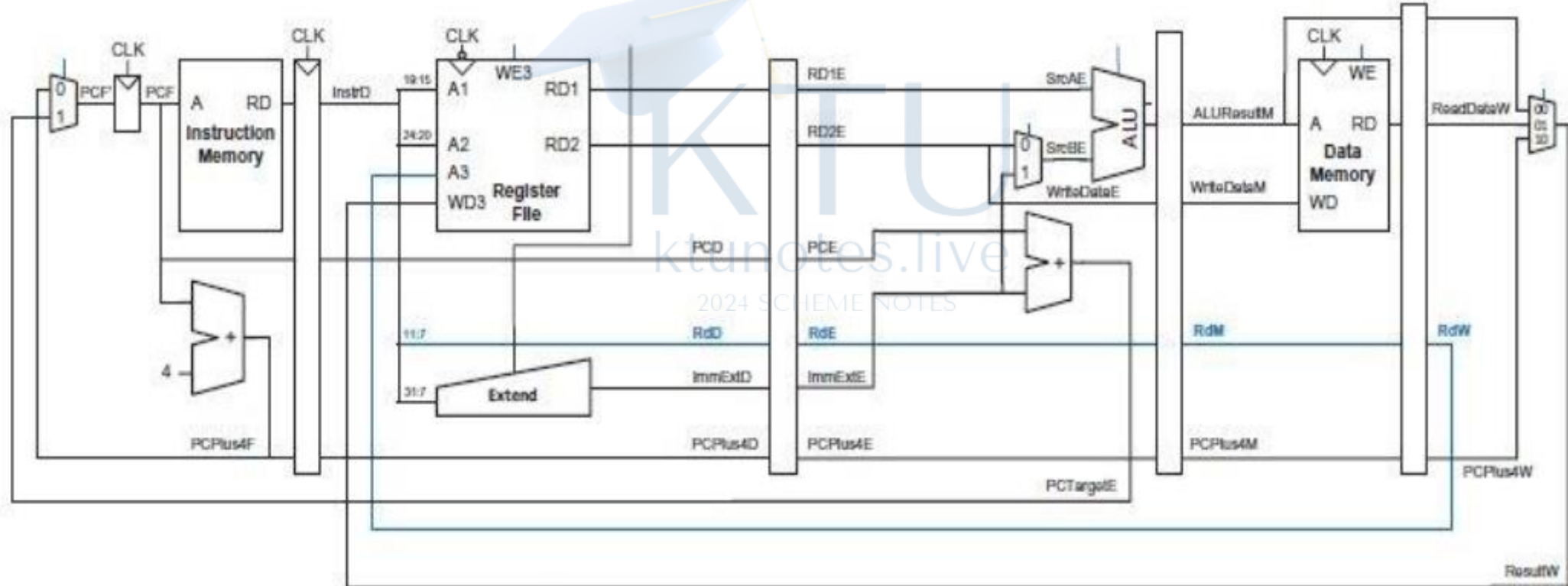


Figure 7.50 Corrected pipelined datapath

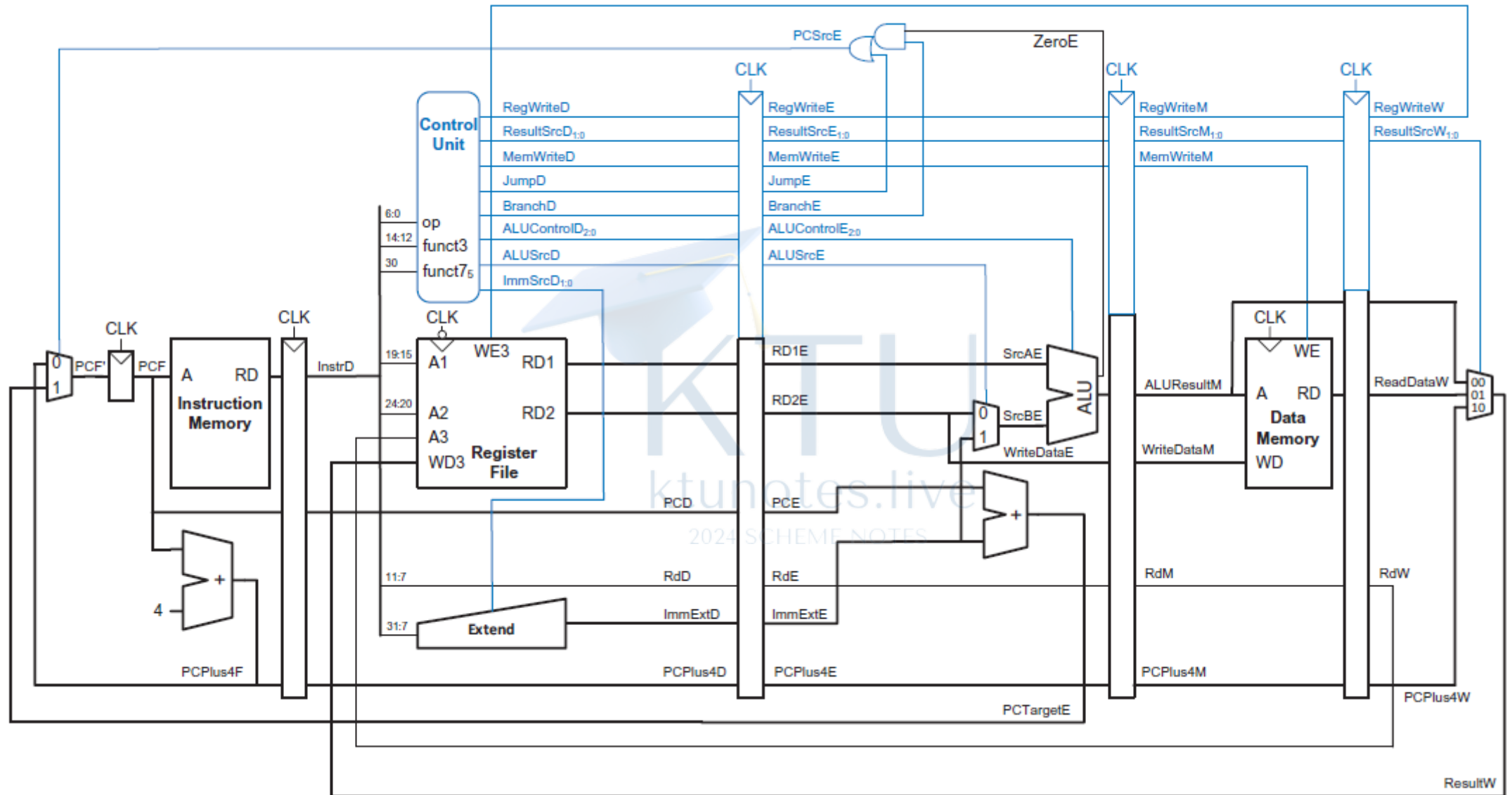
Blue Path	Purpose
EX/MEM → ALU	Forwarding to avoid stalls
MEM/WB → ALU	Forwarding to avoid stalls
Branch target → PC	Correct branch execution
ResultW → Register File	Write-back stage

PCF → Program Counter in Fetch stage

PCF' → The next value that will be loaded into PC

Pipelined Control

- The pipelined processor uses the same control signals as the single-cycle processor and, therefore, has the same control unit.
- The control unit examines the **op**, **funct3**, and **funct7** fields of the instruction in the Decode stage to produce the control signals. For the single-cycle processor.
- These control signals must be pipelined along with the data so that they remain synchronized with the instruction.
- RegWrite must be pipelined into the Write back stage before it feeds back to the register file, just as Rd was pipelined.
- In addition to R-type ALU instructions, lw, sw, and beq, this pipelined processor also supports jal and I-type ALU instructions.



Hazards

- In a pipelined system, multiple instructions are handled concurrently.
- When one instruction is *dependent* on the results of another that has not yet completed, a *hazard* occurs.
- The register file is written during the first half of the cycle and read during the second half of the cycle, so a register can be written and read back in the same cycle without introducing a hazard.

ktunotes.live

2024 SCHEME NOTES

Hazards

- Hazards are classified as data hazards or control hazards.
- A data hazard occurs when an instruction tries to read a register that has not yet been written back by a previous instruction.
- A control hazard occurs when the decision of what instruction to fetch next has not been made by the time the fetch takes place.

ktunotes.live

2024 SCHEME NOTES

Data hazards-RAW,WAR,WAW

✓ RAW – Read After Write (True Dependency)

An instruction needs to **read a register before a previous instruction writes to it.**

add x5, x1, x2 # x5 = x1 + x2

sub x6, x5, x3 # needs x5

○ WAR – Write After Read (Anti Dependency)

A later instruction writes to a register before an earlier instruction reads it.

- **But in RISC-V 5-stage pipeline WAR cannot occur:** ordering prevents WAR

○ WAW – Write After Write (Output Dependency)

- Two instructions write to the same register in overlapping manner.

- **But in RISC-V 5-stage pipeline: WAW cannot occur** because: writes occur in correct sequence

Read after write (RAW) hazard

- A Read-After-Write (RAW) hazard, or true dependency, occurs in computer pipelines when an instruction requires data from a previous instruction that has not yet completed writing it, causing the reading of stale or incorrect data

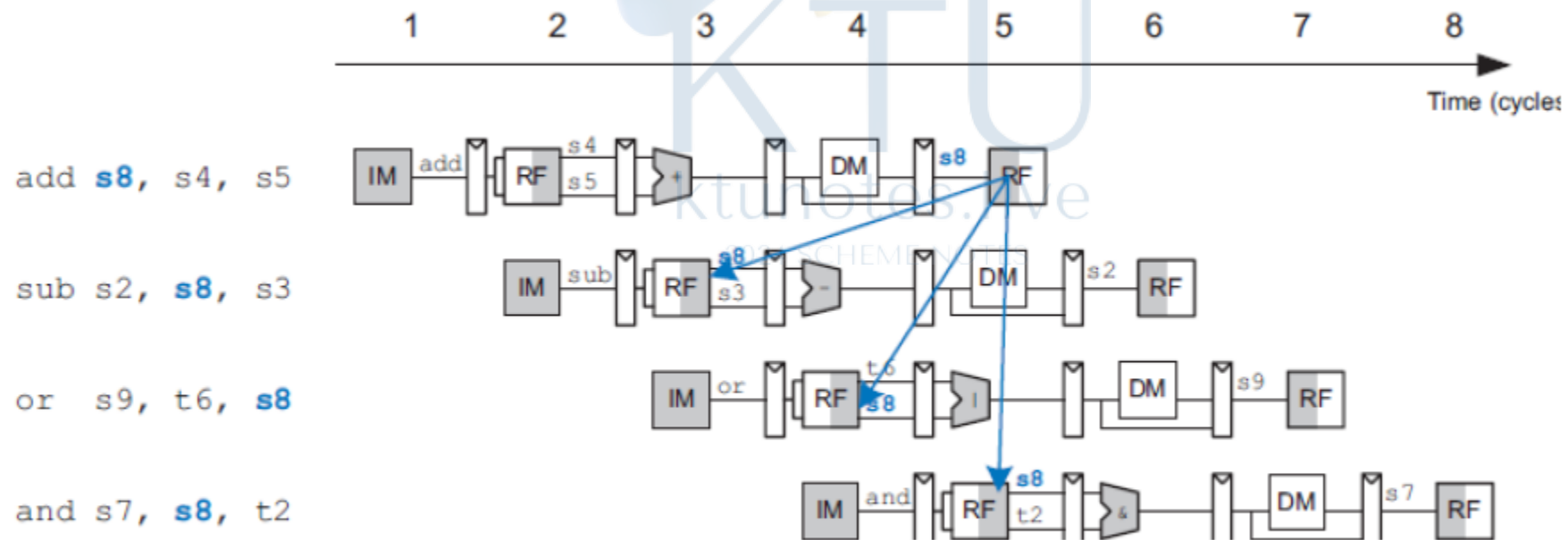


Figure 7.52 Abstract pipeline diagram illustrating hazards

RAW Hazard – Solution1

- A software solution would be to require the programmer or compiler to insert nop instructions between the add and sub instructions so that the dependent instruction does not read the result (s8) until it is available in the register file.

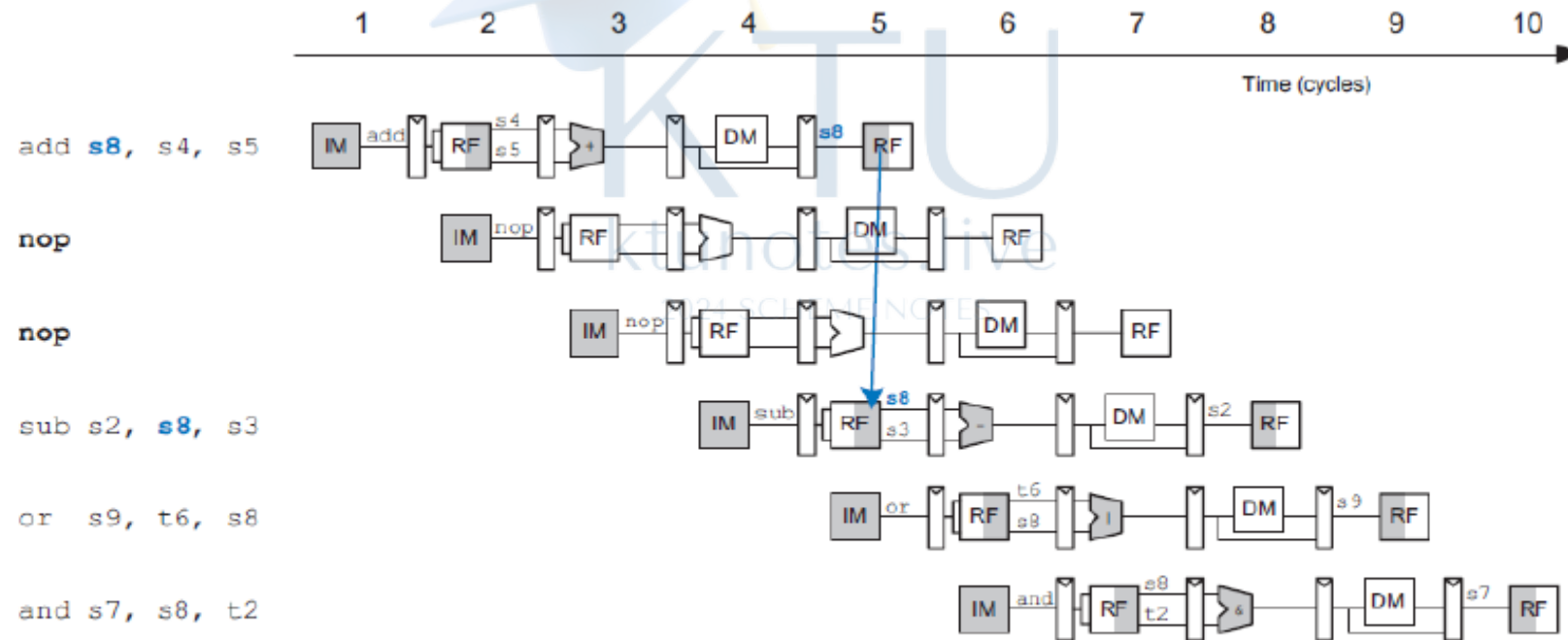


Figure 7.53 Solving data hazard with nops

RAW Hazard – Solution2

- *software interlock* complicates programming and degrades performance, so it is not ideal.
- Some data hazards can be solved by *forwarding* (also called *bypassing*) a result from the Memory or Write back stage to a dependent instruction in the Execute stage.
- This requires adding multiplexers in front of the ALU to select its operands from the register file or the Memory or Write back stage.

2024 SCHEME NOTES

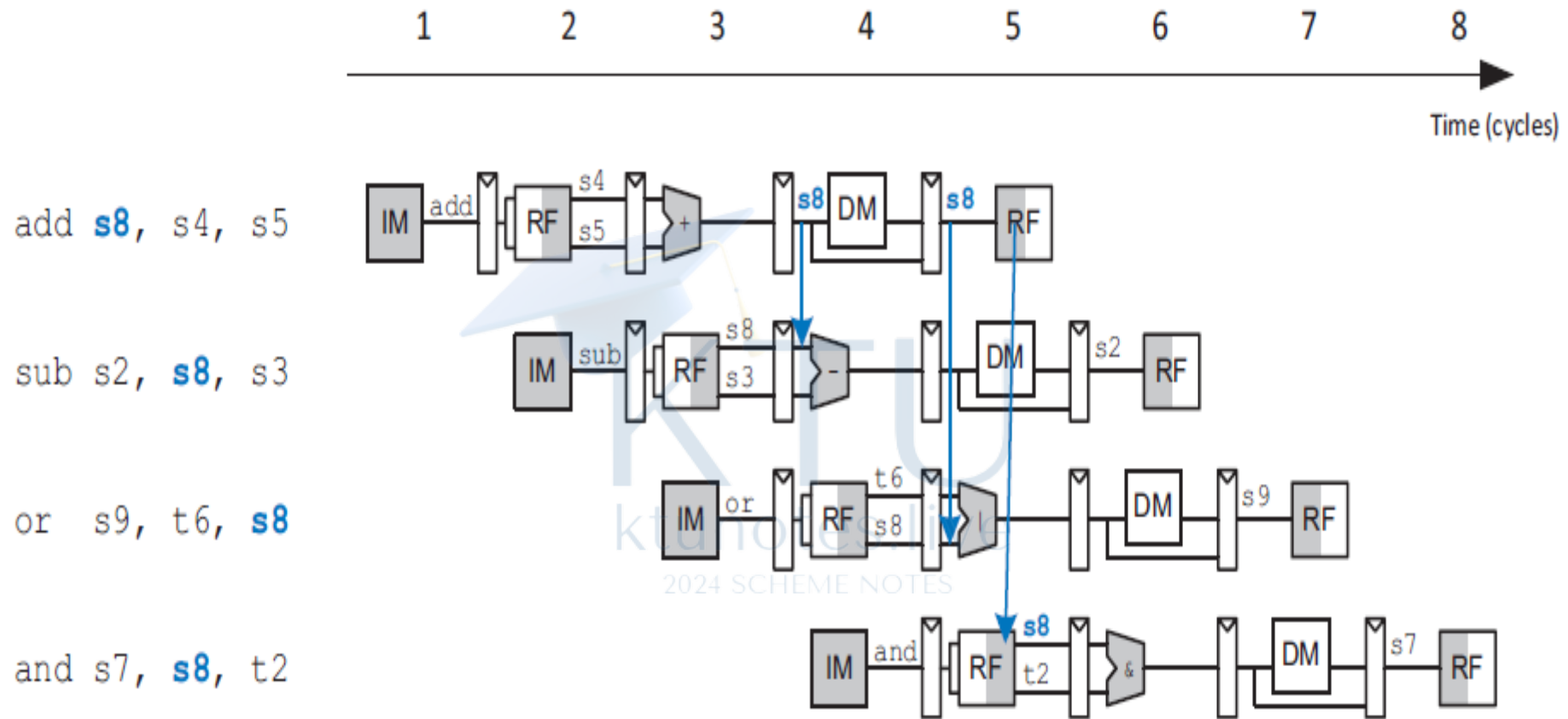
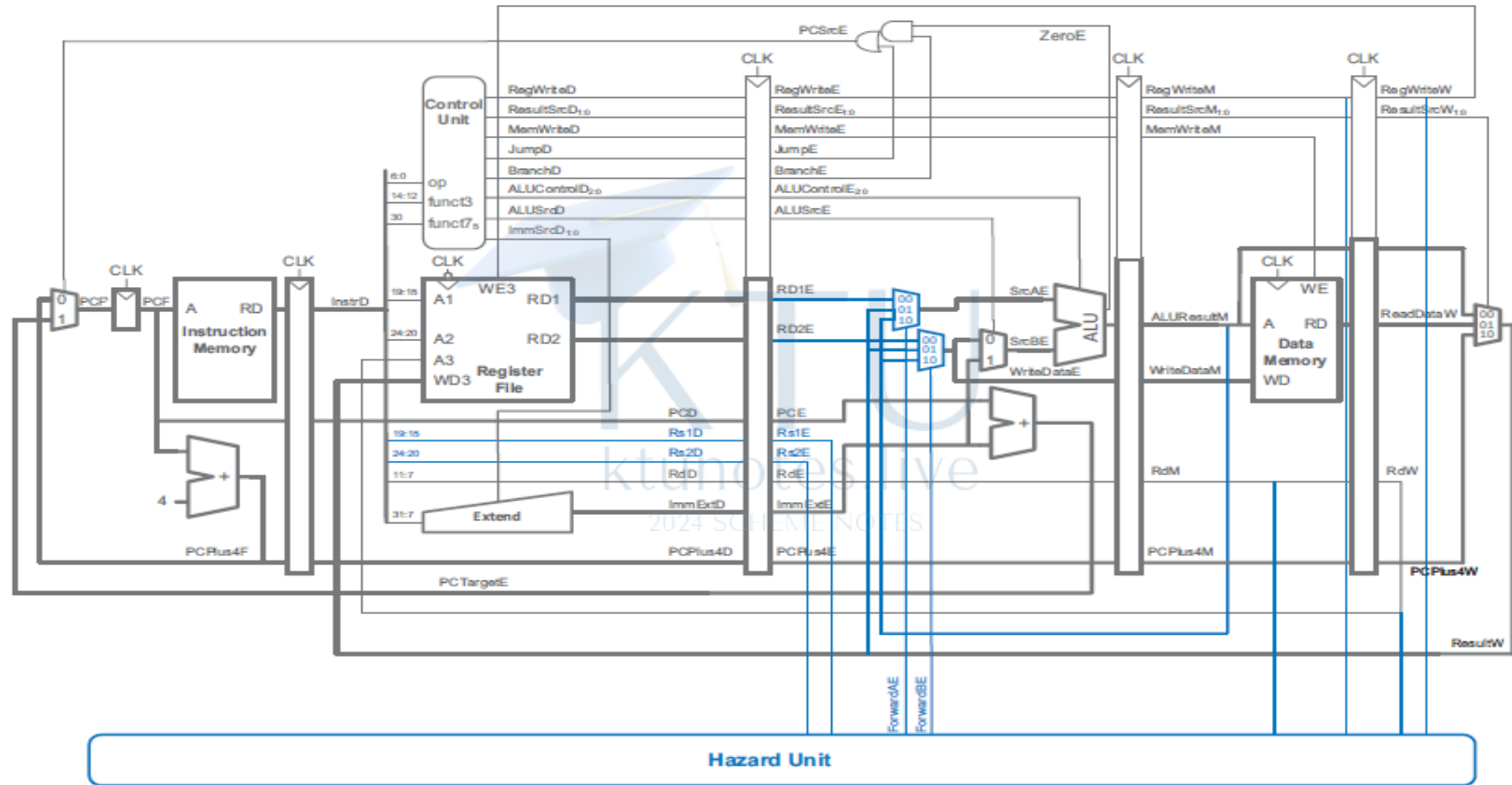


Figure 7.54 Abstract pipeline diagram illustrating forwarding

Pipelined processor with forwarding to solve some data hazards



- Forwarding is necessary when an instruction in the Execute stage has a source register matching the destination register of an instruction in the Memory or Write back stage.
- It adds a **Hazard Unit** and two **forwarding multiplexers**.
- The hazard detection unit receives the two source registers from the instruction in the Execute stage, $Rs1E$ and $Rs2E$, and the destination registers from the instructions in the Memory and Write back stages, RdM and RdW .
- It also receives the $RegWrite$ signals from the Memory and Writeback stages ($RegWriteM$ and $RegWriteW$) to know whether the destination register will actually be written .
- The Hazard Unit computes control signals for the forwarding multiplexers to choose operands from the register file or from the results in the Memory or Writeback stage ($ALUResultM$ or $ResultW$).
- The Hazard Unit should forward from a stage if that stage will write a destination register *and* the destination register matches the source register.

- the function of the forwarding logic for SrcAE (ForwardAE) and SrcBE (ForwardBE) is identical except that it checks Rs2E instead of Rs1E.

```
if      ((Rs1E == RdM) & RegWriteM) & (Rs1E != 0) then // Forward from Memory stage
    ForwardAE = 10
else if ((Rs1E == RdW) & RegWriteW) & (Rs1E != 0) then // Forward from Writeback stage
    ForwardAE = 01
else    ForwardAE = 00                                // No forwarding (use RF output)
```

ktunotes.live

2024 SCHEME NOTES

Solving Data Hazards with Stalls

- Forwarding is sufficient to solve RAW data hazards when the result is computed in the Execute stage of an instruction because its result can then be forwarded to the Execute stage of the next instruction.
- Unfortunately, the lw instruction does not finish reading data until the end of the Memory stage, so its result cannot be forwarded to the Execute stage of the next instruction.
- the lw instruction has a *two-cycle latency* because a dependent instruction cannot use its result until two cycles later.
- A solution is **to stall** the pipeline, holding up operation until the data is available.

Solving Data Hazards with Stalls

- stalling a stage is performed by disabling its pipeline register (i.e., the register to the left of a stage) so that the stage's inputs do not change.
- When a stage is stalled, all previous stages must also be stalled so that no subsequent instructions are lost.
- The pipeline register directly after the stalled stage must be cleared (flushed) to prevent **false, invalid, or incorrect data** from propagating forward.
- Stalls degrade performance, so they should be used only when necessary

Solving Data Hazards with Stalls

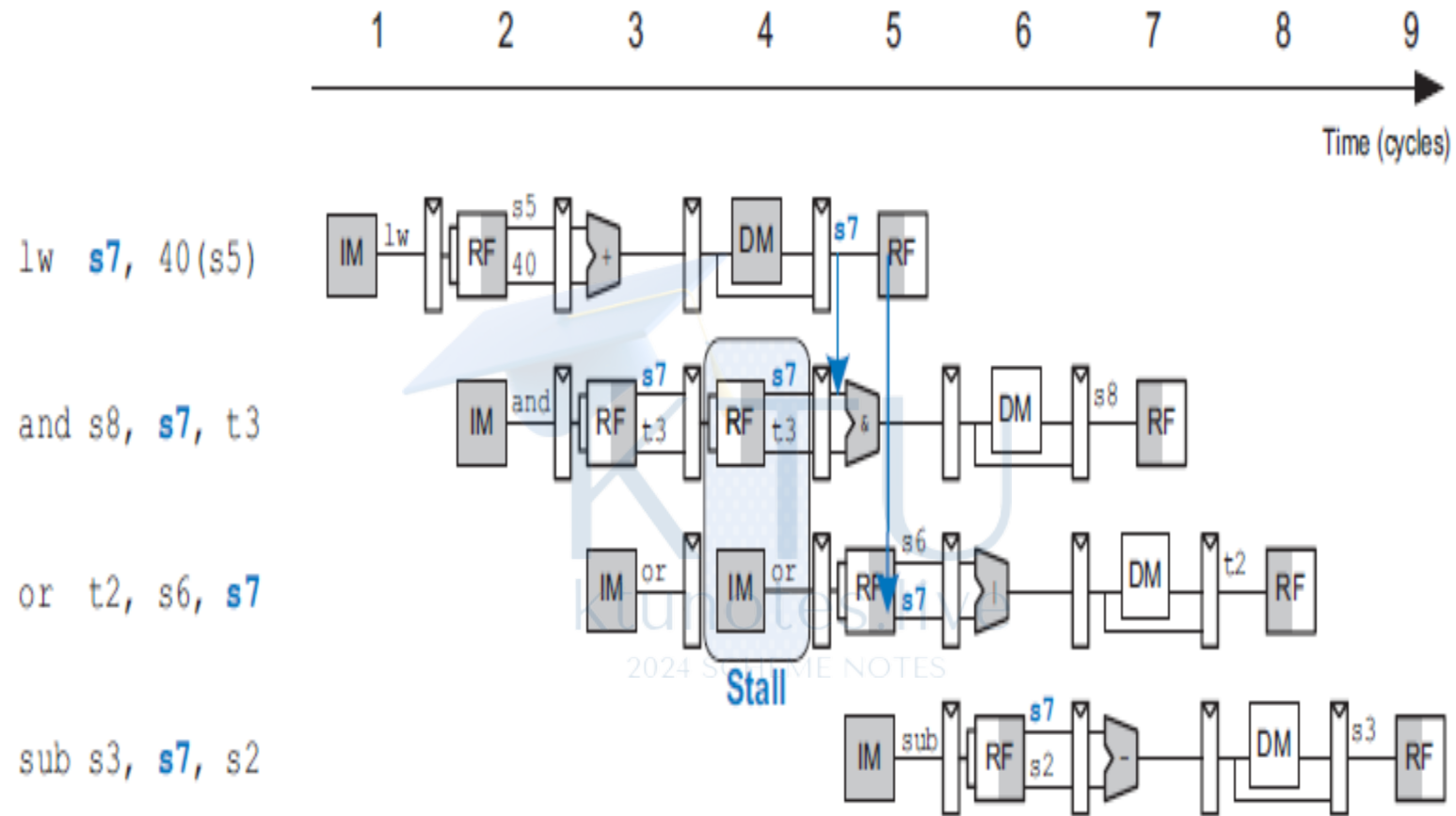


Figure 7.57 Abstract pipeline diagram illustrating stall to solve hazards

Solving Control Hazards

- The beq instruction presents a control hazard: the pipelined processor does not know what instruction to fetch next because the branch decision has not been made by the time the next instruction is fetched.
- One mechanism for dealing with this control hazard is to stall the pipeline until the branch decision is made (i.e., $PCSrcE$ is computed). Because the decision is made in the Execute stage,
 - the pipeline would have to be stalled for two cycles at every branch.
 - This would severely degrade the system performance.

Solving Control Hazards

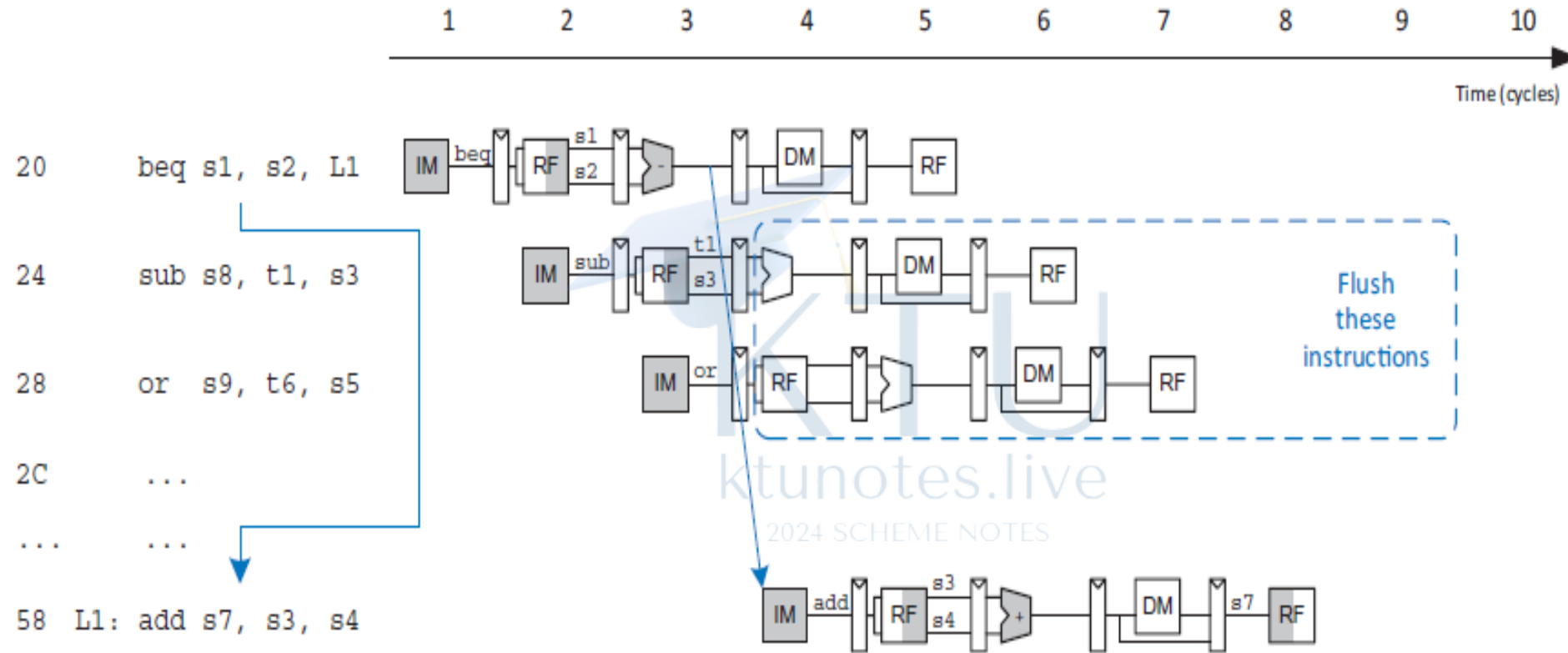


Figure 7.59 Abstract pipeline diagram illustrating flushing when a branch is taken

Solving Control Hazards

- An alternative to stalling the pipeline is **to predict** whether the branch will be taken and begin executing instructions based on the prediction.
- Once the branch decision is available, the processor can throw out the instructions if the prediction was wrong.
- the processor predicts that branches are not taken and simply continues executing the program in order until *PCSrcE* is asserted to select the next PC from *PCTargetE* instead.
- If the branch should have been taken, then the two instructions following the branch must be *flushed* (discarded) by clearing the pipeline registers for those instructions.
- These wasted instruction cycles are called the *branch mis_prediction penalty*.

Performance Analysis

- The pipelined processor ideally would have a CPI of 1 because a new instruction is *issued*—that is, fetched—every cycle.
- However, a stall or a flush wastes 1 to 2 cycles, so the CPI is slightly higher and depends on the specific program being executed.





2024 SCHEME

KTUNOTES.LIVE

SCAN THE QR CODES

join our WhatsApp group



For WhatsApp group

get into our website



website for more materials

CLICK THE LOGO TO JOIN



OUR TELEGRAM CHANNEL

for notes and discussion and updates



OUR WHATSAPP GROUP

for note updates and notifications



OUR WEBSITE

get in and download all 2024 materials